

UNIVERSITY OF MYSORE

PROJECT REPORT

**DEVELOPMENT OF LEVELUPX:
A WEB-BASED GAMIFIED PRODUCTIVITY
AND
FOCUS MANAGEMENT PLATFORM**

A project report submitted in partial fulfillment of the requirements
for the award of the degree of

BACHELOR OF COMPUTER APPLICATIONS

Submitted by

Student name: _____

Registration / enrollment number: _____

Under the guidance of

Project guide: _____

Designation: _____

Study centre / institution: _____

Academic year: _____

DECLARATION BY THE STUDENT

I hereby declare that the project report entitled “Development of LevelUpX: A Web-Based Gamified Productivity and Focus Management Platform” is a record of the project work carried out by me as part of the Bachelor of Computer Applications programme. I developed and studied the project independently under the guidance named on the title page. The technical description, results, and limitations in this report are based on the source code, configuration, documentation, and verification evidence available in the project workspace.

I further declare that this report has not been submitted, either in full or in part, for the award of any other degree or diploma. Wherever I used published ideas, standards, or technical documentation, I have acknowledged the source in the text and bibliography. I have not presented planned features, external services, or historical test records as if they were current production results.

Place: _____

Submission date: _____

Signature of the Student: _____

ACKNOWLEDGEMENT

I express my sincere gratitude to my project guide (name: _____) for the guidance, review, and encouragement provided during this project. The feedback helped me examine the system more carefully and explain my implementation decisions in a structured academic form.

I thank the faculty members and coordinators of the study centre / institution (name: _____) and the University of Mysore for giving me the opportunity to complete this final-year project. The BCA subjects on programming, database management, web development, software engineering, system analysis, and testing gave me the foundation required to design and implement LevelUpX.

I am also thankful to my family and friends for their patience and support during development, repeated testing, documentation, and revision. Finally, I acknowledge the maintainers of the open-source frameworks and official documentation referenced in this report. Their work made it possible for me to study and apply modern web-development practices in a student project.

ABSTRACT

Many task-management tools record what a user plans to do, but they provide limited immediate feedback after the work is completed. LevelUpX was developed as a web-based gamified productivity platform that connects ordinary work with quests, timed focus sessions, experience points, coins, levels, achievements, analytics, and optional social accountability. The system is intended for students, professionals, and personal-productivity users. It includes separate user and administrative capabilities while keeping PostgreSQL as the main source of truth.

I implemented the project as a TypeScript modular monolith. The frontend uses Next.js 16, React 19, Tailwind CSS, Zustand, Recharts, Chart.js, Framer Motion, IndexedDB, and a service worker. The backend uses Express, Zod, Prisma, PostgreSQL, Socket.IO, and optional Redis support. The main functions include verified registration, ordinary and privileged authentication, projects and quests, focus-session tracking, reward ledgers, automatic UTC streaks, prerequisite skills, achievements, guilds, recurring weekly team quests, accepted-peer focus presence, leaderboards, bounded rule-based productivity insights, notifications, account controls, automated abuse signals, CSV/PDF reports, a dedicated administrator login entry, and an administrator control room. A transaction-based quest-completion flow updates the quest, reward ledgers, profile balances, level, and streak state together. Offline quick capture uses IndexedDB and an idempotent client request identifier when the queued action is replayed.

On 12 August 2026, I reran the available non-database verification. All 270 tests in 53 files in the default backend Vitest suite, 10 frontend tests in 6 files, and 5 HTTP integration tests in 1 file passed. Backend and frontend type checks, zero-warning lint commands, and production builds also passed; the Next.js build generated 25 static pages. The 16 database tests in 4 files were collected but skipped because no isolated test database was configured, and Playwright was not rerun after the adherence changes. The current version is a substantial working local implementation, but public hosting, real production email and push credentials, managed backups, physical passkey enrolment, cross-browser testing, and formal usability studies remain future work.

Keywords: gamification, productivity, quests, focus sessions, PWA, TypeScript, PostgreSQL, Socket.IO

TABLE OF CONTENTS

DECLARATION BY THE STUDENT.....	i
ACKNOWLEDGEMENT.....	ii
ABSTRACT.....	iii
TABLE OF CONTENTS.....	iv
LIST OF FIGURES.....	ix
LIST OF TABLES.....	x
ABBREVIATIONS.....	xii
OPERATIONAL DEFINITIONS.....	xiii
INTRODUCTION.....	1
1.1 Introduction to the Project.....	1
1.1.1 Problem Definition.....	2
1.1.2 Existing Approaches.....	3
1.1.3 Proposed System.....	3
1.1.4 Company Profile.....	4
1.2 Objectives of the Project Work.....	4
1.3 Scope of the Project Work.....	6
1.3.1 Included Scope.....	6
1.3.2 Excluded or Deferred Scope.....	6
1.3.3 Feasibility Study.....	7
1.4 Project Report Outline.....	8
LITERATURE SURVEY/REVIEW.....	10
2.1 System Study.....	10
2.2 Review of Literature.....	11
2.2.1 Study of Gamification.....	11
2.2.2 Goal Setting and Motivation.....	11
2.2.3 Focus Sessions and Break Regulation.....	12

2.2.4 Personal Informatics and Analytics.....	12
2.2.5 Workload and Burnout Terminology.....	13
2.2.6 Offline and Real-Time Foundations.....	13
2.2.7 Consolidated Literature Review.....	14
2.3 Comparison of Literature.....	15
2.4 Official Technology Review.....	16
2.5 Literature Gap and Project Position.....	18
SYSTEM REQUIREMENTS SPECIFICATION.....	19
3.1 Functional Requirements.....	19
3.2 Non-Functional Requirements.....	22
3.3 Hardware Requirements.....	24
3.4 Software Requirements.....	25
3.5 User Roles and Privileges.....	26
3.6 Data Requirements.....	26
3.7 Interface Requirements.....	27
3.8 Assumptions, Constraints, and Acceptance.....	27
SYSTEM DESIGN.....	29
4.1 Design Overview.....	29
4.2 System Architecture.....	30
4.2.1 Frontend Layer.....	30
4.2.2 Backend Layer.....	30
4.2.3 Data, Realtime, and Job Layers.....	31
4.3 Data Flow Diagrams.....	33
4.4 Use Case Diagram.....	35
4.4.1 Main Success Scenario: Complete a Quest.....	35
4.4.2 Alternate and Failure Scenarios.....	36
4.5 Class Diagram.....	36

4.6 Sequence Diagram.....	37
4.6.1 Privileged Authentication.....	37
4.6.2 Quest Completion.....	38
4.7 Activity Diagram.....	39
4.8 E-R Diagram.....	40
4.9 Modules of the Project.....	42
4.10 Offline and Synchronization Design.....	45
4.11 API and Error Design.....	46
4.12 Security Design.....	46
4.13 Deployment and Operations Design.....	48
4.14 Design Rationale and Trade-offs.....	49
IMPLEMENTATION.....	50
5.1 Steps for Implementation.....	50
5.2 Implementation Issues.....	51
5.3 Algorithms.....	53
5.3.1 Quest Reward Calculation.....	53
5.3.2 UTC Streak Maintenance.....	54
5.3.3 Level Derivation.....	54
5.3.4 Skill Prerequisite Resolution.....	55
5.3.5 Idempotent Offline Replay.....	55
5.3.6 Recurring Job Execution.....	56
5.4 Implementation Strategy.....	57
5.5 Frontend Implementation.....	58
5.6 Backend Implementation.....	60
5.7 Database and Migration Implementation.....	61
5.7.1 Ownership and Relational Modeling.....	61
5.7.2 Transactions and Concurrency.....	61

5.7.3 Migration and Data-Safety Rules.....	62
5.8 Authentication and Security Implementation.....	63
5.9 Analytics and Insight Implementation.....	63
5.10 Realtime and Notification Implementation.....	64
5.11 Navigation and Interface Implementation.....	65
TESTING.....	66
6.1 Test Environment.....	67
6.2 Unit Testing of Modules.....	68
6.3 Integration Testing of Modules.....	70
6.4 System Testing.....	71
6.5 Functional Testing.....	72
6.6 Results.....	74
6.7 Security-Focused Testing.....	75
6.8 Test Result Analysis.....	75
6.9 Testing Limitations.....	76
6.10 Acceptance Summary.....	77
RESULTS AND ANALYSIS.....	78
7.1 Result Overview.....	78
7.2 Results by Functional Module.....	78
7.3 Objective Achievement Analysis.....	81
7.4 Quantitative Implementation Evidence.....	82
7.5 Result Interpretation.....	83
7.5.1 Productivity Result.....	83
7.5.2 Integrity and Security Result.....	83
7.5.3 Offline and Realtime Result.....	83
7.5.4 Analytics and Workload Result.....	83
7.6 Current Limitations.....	84

7.7 Requirements Traceability Summary.....	85
7.8 Discussion.....	86
CONCLUSION AND FUTURE WORK.....	87
8.1 Major Contributions.....	88
8.1.1 Lessons Learned.....	89
8.2 Future Enhancements.....	89
8.2.1 Suggested Evaluation Plan.....	91
8.3 Bibliography.....	92
ORIGINAL SYNOPSIS CONFORMANCE.....	95
A.1 Objective-to-Implementation Matrix.....	95

LIST OF FIGURES

Figure 1.1 Project context and expected users.....	2
Figure 2.1 Literature concepts mapped to LevelUpX features.....	10
Figure 4.1 High-level system architecture.....	29
Figure 4.2 Functional module map.....	31
Figure 4.3 Context-level data flow diagram.....	33
Figure 4.4 Level-1 data flow diagram.....	34
Figure 4.5 Use-case overview.....	35
Figure 4.6 UML class and dependency diagram.....	36
Figure 4.7 Privileged authentication sequence.....	37
Figure 4.8 Quest completion sequence.....	38
Figure 4.9 Quest lifecycle activity diagram.....	39
Figure 4.10 Core entity-relationship diagram.....	40
Figure 4.11 Offline quest capture and replay flow.....	45
Figure 4.12 Local deployment topology.....	48
Figure 5.1 Reward calculation stages.....	53
Figure 5.2 Background job and notification flow.....	56
Figure 5.3 Implemented navigation structure.....	65
Figure 6.1 Testing layers used in the project.....	67

LIST OF TABLES

Table P.1 Abbreviations used in the report.....	xii
Table 1.1 Existing approaches and LevelUpX response.....	3
Table 1.2 Project objectives and measurable outcomes.....	5
Table 1.3 Scope boundaries.....	7
Table 1.4 Feasibility summary.....	8
Table 2.1 Selected literature and design implications.....	14
Table 2.2 Comparison of reviewed approaches.....	15
Table 2.3 Official technical sources consulted.....	17
Table 3.1 Functional requirements.....	19
Table 3.2 Non-functional requirements.....	22
Table 3.3 Hardware requirements.....	24
Table 3.4 Software requirements.....	25
Table 3.5 User roles and privileges.....	26
Table 4.1 Backend module responsibilities.....	31
Table 4.2 Core database entities.....	41
Table 4.3 Module inputs, processing, outputs, and boundaries.....	43
Table 4.4 API response and error conventions.....	46
Table 4.5 Security controls and their purpose.....	46
Table 5.1 Implementation issues and resolutions.....	51
Table 5.2 Quest difficulty multipliers.....	53
Table 5.3 Background jobs.....	56
Table 5.4 Implementation stages.....	58
Table 5.5 Frontend responsibilities.....	59
Table 5.6 Backend layer responsibilities.....	60
Table 5.7 Critical relational invariants and enforcement.....	62
Table 6.1 Current verification environment.....	67

Table 6.2 Representative unit tests.....	68
Table 6.3 Integration and database test evidence.....	70
Table 6.4 Functional test cases.....	72
Table 6.5 Verification command results on 12 August 2026.....	74
Table 6.6 Testing limitations.....	76
Table 7.1 Implemented results by module.....	78
Table 7.2 Objective achievement analysis.....	81
Table 7.3 Current limitations and their effect.....	84
Table 8.1 Major contributions.....	88
Table 8.2 Prioritized future enhancements.....	89
Table A.1 Original synopsis objectives mapped to the implemented system.....	95

ABBREVIATIONS

Table P.1 summarizes abbreviations used in the report.

Table P.1 Abbreviations used in the report

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CI	Continuous Integration
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, and Delete
CSV	Comma-Separated Values
DFD	Data Flow Diagram
ER	Entity-Relationship
HTTP	Hypertext Transfer Protocol
IndexedDB	Browser database for structured client-side data
JWT	JSON Web Token
ORM	Object-Relational Mapping
PDF	Portable Document Format
PWA	Progressive Web Application
RBAC	Role-Based Access Control
REST	Representational State Transfer
SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language
TLS	Transport Layer Security
UI	User Interface
UX	User Experience
UUID	Universally Unique Identifier
WebAuthn	Web Authentication API
XP	Experience Points

OPERATIONAL DEFINITIONS

- Quest: a user-created unit of planned work with a title, difficulty, estimated duration, status, and calculated reward.
- Focus session: a timed work or break interval that may be linked to a quest and can be paused, resumed, annotated, completed, or stopped.
- Experience points (XP): a non-monetary progression value recorded in an append-only transaction history through current workflows and used to derive the displayed level.
- Coin: an internal virtual balance earned through supported actions and spent on cosmetics or user-defined rewards; it has no cash value.
- Guild: a public or private group whose members can participate in team quests and view a scoped leaderboard.
- Insight: rule-based or provider-assisted feedback derived from stored productivity records; it is not a medical diagnosis.
- Privileged user: an ADMIN or SUPER_ADMIN account that must satisfy additional device, code, and passkey checks.

CHAPTER I

INTRODUCTION

1.1 Introduction to the Project

Students and professionals often divide their work between task lists, timers, calendars, notes, and messaging groups. A task list can record a deadline, but it normally gives little immediate feedback about sustained effort. A timer can record a study interval, but it may not connect the session to the task that was advanced. Social tools can provide encouragement, but their progress records are usually separate from individual planning. I selected this problem because the missing connection between planning, focused execution, reflection, and feedback can make ordinary trackers feel passive after the initial setup.

LevelUpX addresses this gap by treating planned work as a quest. A quest can have a project, priority, difficulty, tags, deadline, recurrence rule, estimated time, subtasks, and reminder. When the user starts and completes valid work, the backend records progress and grants controlled XP and coins. Focus sessions provide timed work and break modes. The same profile then connects quests with levels, skills, achievements, analytics, optional guild activity, and personal rewards. The system does not assume that a reward automatically creates motivation. Instead, the reward is used as visible feedback while the underlying task remains owned by the user.

The project is a browser-based Progressive Web Application. It can be used from a desktop or mobile browser, and its manifest and service worker provide installable and offline foundations. Offline support is deliberately limited: quick-capture quest creation can be queued in IndexedDB and replayed later. Other features still depend on the API and database. This boundary is important because it avoids claiming that the complete application works without a connection. Figure 1.1 illustrates project context and expected users.

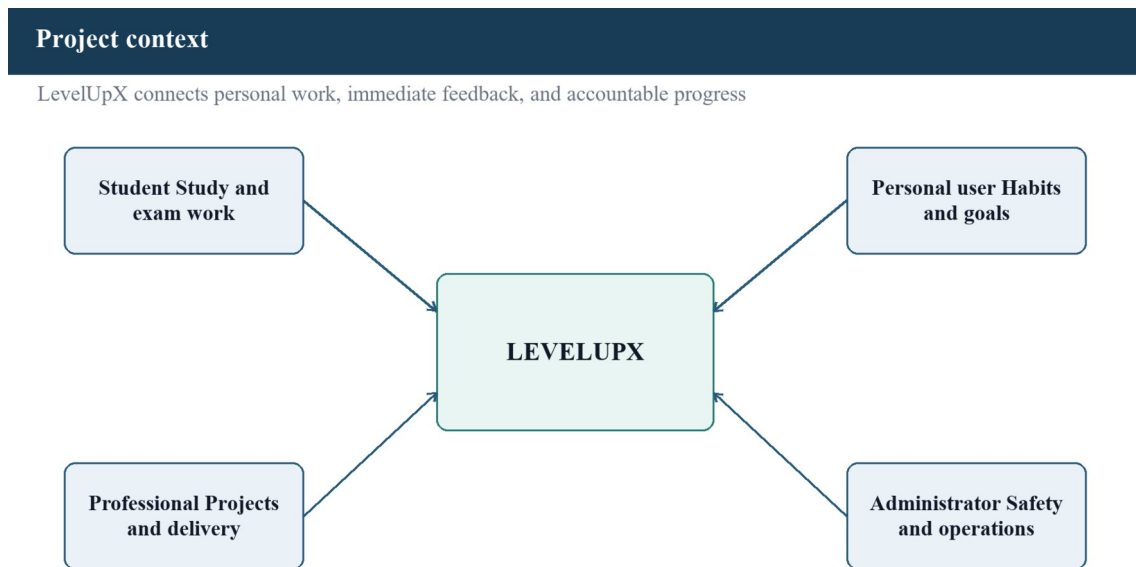


Figure 1.1 Project context and expected users

1.1.1 Problem Definition

The problem addressed by this project is the separation between planning, focused work, progress feedback, and accountability in common productivity methods. Physical planners and basic note applications require manual updates and provide no calculated progression. Simple digital task lists can store and complete items, but they may not explain how much focused time was spent, preserve a reward history, or connect the result to a longer-term skill path. Timer applications can support concentration but often treat the timer as an isolated activity. These limitations do not make existing tools unusable; they define the opportunity explored by LevelUpX.

A second problem is data consistency when several features react to one completion. A completed quest may change its state, add XP, add coins, change a level, create ledger rows, unlock an achievement, update a leaderboard, and produce a notification. If each update is performed without an atomic boundary, a partial failure can leave the visible

profile inconsistent. The project therefore required a transaction-based completion design and an idempotency rule that prevents the same quest from paying twice.

A third problem concerns trust and scope. Social and administrative features introduce access control requirements beyond those of a single-user timer. Ordinary users must access only their own records or authorized guild data. Administrative operations require stronger verification because they can change roles, economy settings, moderation outcomes, and system configuration. The design therefore separates ordinary user authentication from a staged privileged path.

1.1.2 Existing Approaches

Table 1.1 summarizes existing approaches and LevelUpX response.

Table 1.1 Existing approaches and LevelUpX response

Approach	Useful capability	Observed limitation for this project	LevelUpX response
Paper planner	Flexible personal planning	Manual calculation; no synchronized history	Structured quests, ledgers, and exports
Basic task list	Quick creation and completion	Limited connection to focus and progression	Quest lifecycle linked to focus and rewards
Standalone timer	Time-boxed work	Session may not identify the task advanced	Optional quest link, goal, notes, and history
Habit tracker	Streak and repetition feedback	Often narrow for projects and teamwork	Projects, recurrence, skills, guilds, and analytics
Group chat	Social encouragement	Progress evidence is informal and scattered	Guild membership, team quests, and scoped ranking

1.1.3 Proposed System

The proposed system combines user, productivity, progression, social, insight, and administration modules in one web application. A new user verifies an email address, completes onboarding, chooses a productivity mode and character class, and creates a first quest. The daily workflow starts from a dashboard that highlights an active or urgent

quest, weekly completion, focus history, streak, and recent notifications. The user can then plan detailed quests, run a focus session, review the reward outcome, and use analytics or insights for reflection.

The backend is implemented as a modular monolith. Each business module exposes routes and normally separates controller, service, repository, validation, and type responsibilities. PostgreSQL is the durable source of truth, while Redis is optional for rate limits, short-lived leaderboard caching, and distributed job locks. Socket.IO delivers selected real-time events, but important mutations still pass through the REST API and database. A separate scheduler process runs recurring reports, weekly team-quest materialization, leaderboard refreshes, reminder checks, rule-based workload signals, scheduled insights, and daily digests.

1.1.4 Company Profile

This project was completed as an individual academic project and was not developed for a company. Therefore, a company profile is not applicable. The relevant institutional details are the BCA programme, the University of Mysore, and the study centre or institution named on the cover page. If the university requires a separate organization profile, the actual organization information must be supplied before submission.

1.2 Objectives of the Project Work

Aim: To design, implement, and verify a web-based platform that connects task planning, focused work, gamified feedback, analytics, and controlled social accountability.

1. Develop verified registration, login, recovery, session, and role-based access-control functions.
2. Allow a user to create projects and quests with difficulty, priority, tags, recurrence, deadlines, estimates, and lifecycle actions.
3. Implement timed focus sessions with work and break modes, pause/resume, notes, optional quest linkage, and history.
4. Record XP, coins, automatic UTC streaks, levels, prerequisite-based skills, achievements, cosmetics, and reward transactions without duplicate quest payouts.
5. Provide analytics and rule-based insights that support reflection without presenting health guidance as diagnosis.
6. Support guilds, recurring weekly team quests, leaderboards, accepted-partner live focus presence, accountability connections, notifications, and abuse reporting.

7. Provide a dedicated privileged login and administrative tools for users, roles, economy settings, moderation, automatic abuse signals, CSV/PDF reports, product events, system health, and security operations.
8. Support installable PWA behavior and limited offline quick capture with safe replay.
9. Apply input validation, centralized error handling, rate limits, secure session handling, and staged privileged authentication.
10. Verify the implementation through unit, integration, database, browser, static-analysis, and build checks as far as the available environment permits. Table 1.2 summarizes project objectives and measurable outcomes.

Table 1.2 Project objectives and measurable outcomes

Objective area	Evidence of completion
Identity	Registration, email verification, login, recovery, refresh, logout, session review, and privileged stages are present.
Productivity	Projects, quests, focus modes, notes, history, recurrence, reminders, and offline quick capture are implemented.
Progression	XP and coin ledgers, automatic UTC streaks, levels, achievements, prerequisite-based skills, cosmetics, marketplace, and custom rewards are implemented.
Social	Guilds, invitation codes, recurring weekly team quests, accepted-partner focus presence, leaderboards, and membership-authorized real-time rooms are present.
Operations	Admin control room, reports, product funnel, health, metrics, logging, Prometheus configuration, and audit records are present.
Verification	Current unit, HTTP integration, type-check, lint, and build commands passed; unavailable lanes are clearly identified.

1.3 Scope of the Project Work

1.3.1 Included Scope

- Responsive browser and installable PWA interface for user and administrative workflows.
- User modes for study, work, and personal goals; these are preferences rather than security roles.
- Quest planning, projects, subtasks, recurrence, focus sessions, rewards, analytics, and rule-based insight actions.
- Character classes, prerequisite-based skills, UTC streaks, achievements, cosmetics, marketplace actions, and custom real-life rewards.
- Guilds, recurring weekly team quests, invitation codes, scoped leaderboards, accepted-partner focus presence, accountability relationships, and abuse reporting.
- Email-verification foundations, ordinary optional two-step login, and layered privileged authentication.
- Local Docker topology with PostgreSQL, Redis, Mailpit, scheduler, Prometheus, Alertmanager, Grafana, backend, and frontend.
- Automated unit, integration, database, and Chromium browser test source, with current evidence reported by lane.

1.3.2 Excluded or Deferred Scope

- A native Android or iOS application; the current delivery is a web PWA.
- A public production deployment with a verified domain, TLS, managed secrets, managed database, backups, and restore drills.
- Production email, web-push, on-call, error-tracking, and external uptime-provider accounts.
- A trained machine-learning model or clinical burnout assessment; current suggestions are intentionally rule based.
- Payment processing, subscription billing, enterprise identity, or exchange of coins for real money.
- Formal multi-browser, physical-device, load, penetration, accessibility-user, or large-sample usability studies.

- Complete offline use of all features; only quick-capture quest creation has a queue and replay workflow. Table 1.3 summarizes scope boundaries.

Table 1.3 Scope boundaries

Area	Current boundary	Reason
Platform	Responsive PWA, not native mobile	Maintains one codebase within the project duration
AI	Rules and optional provider contract	No consented dataset for validated model training
Offline	Quest quick capture only	Other modules depend on authoritative server state
Deployment	Complete local Compose topology	Public infrastructure needs external accounts and operator decisions
Testing	Automated suites plus static/build verification	Docker and physical-device lanes depend on available environment
Economy	Virtual, non-cash XP and coins	The project is not a financial or payment system

1.3.3 Feasibility Study

Technical feasibility was supported by the availability of TypeScript across both applications, the established web ecosystem, a relational PostgreSQL design, and local container orchestration. The modular monolith reduced the operational complexity of separate microservices while preserving module boundaries. The current host successfully completed both production builds and the available automated checks. Public production feasibility is not yet demonstrated because final infrastructure, domains, provider credentials, and recovery procedures are outside the local project.

Operational feasibility is reasonable for users familiar with browser-based applications. The navigation groups daily work, progression, social features, intelligence, and system actions. A guided onboarding flow creates the first quest. Administrative operation is more demanding because it uses trusted-device, code, and passkey stages; this additional effort is intentional for higher impact actions. Formal usability testing was not supplied, so the report does not claim that every user will find the interface easy without training.

Economic feasibility is supported by the use of open-source frameworks and local tools. The project can be developed and demonstrated without licence fees. A public installation would still have recurring costs for hosting, email delivery, managed databases, object storage, monitoring, backups, and domain registration. Those costs cannot be estimated accurately until providers and expected usage are selected. Table 1.4 summarizes feasibility summary.

Table 1.4 Feasibility summary

Dimension	Current assessment	Remaining evidence
Technical	Feasible as a local modular-monolith PWA	Hosted reliability, backup, and scaling evidence
Operational	Main workflows are implemented and navigable	Formal usability and support procedure
Economic	Open-source local development is low-cost	Provider selection and operating-cost estimate
Schedule	Current synopsis scope is implemented locally	University submission milestones and sign-off dates

1.4 Project Report Outline

1. Chapter I introduces the problem, objectives, scope, feasibility, and structure of the report.
2. Chapter II reviews gamification, goal-setting, focus-session, personal-informatics, workload, PWA, and real-time literature.
3. Chapter III defines functional, non-functional, hardware, software, role, data, and interface requirements.
4. Chapter IV explains the architecture, data flows, use cases, class structure, sequences, activities, database, modules, security, and deployment design.
5. Chapter V describes the implementation process, important algorithms, validations, technical decisions, and problems encountered.
6. Chapter VI reports the test environment, unit, integration, database, browser, functional, static-analysis, and build evidence.

7. Chapter VII evaluates the implemented results against the stated objectives and records the limits of the evidence.
8. Chapter VIII presents the conclusion, major contributions, future work, lessons learned, and bibliography.

CHAPTER II

LITERATURE SURVEY/REVIEW

2.1 System Study

The system study examined the project as a combination of productivity planning, gamified feedback, personal analytics, focused work, collaboration, and secure web engineering. The purpose of the study was not to prove that one feature automatically changes behaviour. It was to identify the mechanisms, benefits, and limits that should influence the implementation.

The implemented workflow follows a preparation-to-action cycle. The user prepares work as a quest, performs it during one or more focus sessions, receives a recorded outcome, reviews progress, and may choose a next action. This cycle resembles personal-informatics models in which collection and integration must be reliable before reflection is useful. It also follows goal-setting research by encouraging a specific title, estimated time, difficulty, priority, and deadline instead of a vague intention. Figure 2.1 illustrates literature concepts mapped to LevelUpX features.

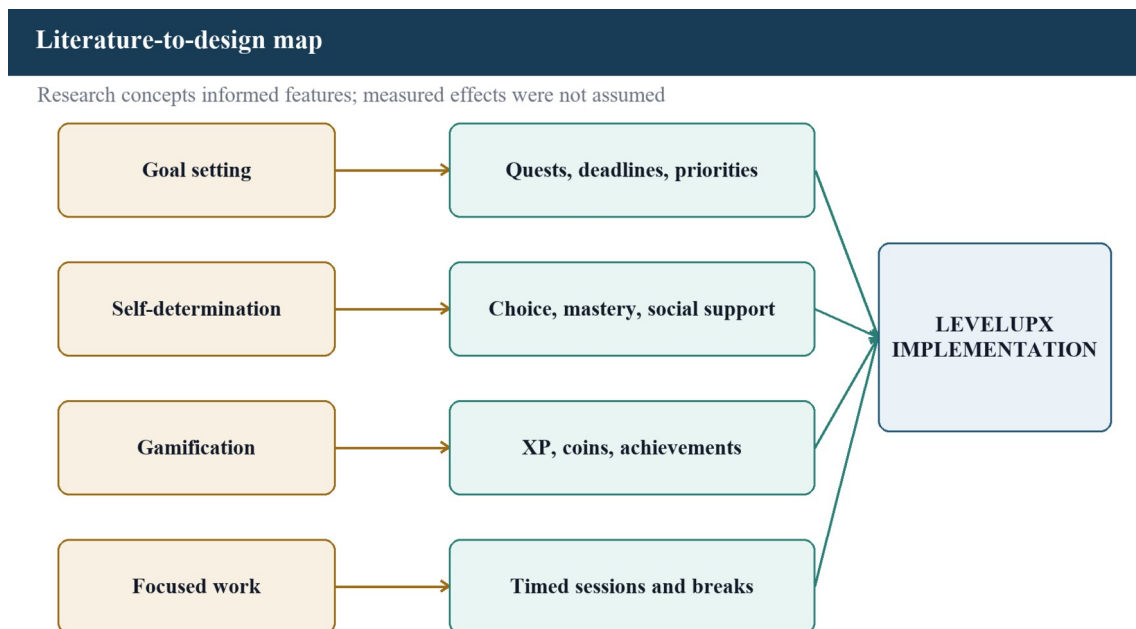


Figure 2.1 Literature concepts mapped to LevelUpX features

2.2 Review of Literature

2.2.1 Study of Gamification

Deterding et al. define gamification as the use of game-design elements in non-game contexts [1]. LevelUpX matches this definition because the underlying activity is real study, work, or personal planning, while XP, coins, levels, classes, achievements, cosmetics, and leaderboards provide a game-influenced representation. The application is not a complete role-playing game and does not replace the user's task with a fictional objective.

Experimental research shows that individual game elements can influence different outcomes. Sailer et al. found that badges, leaderboards, performance graphs, avatars, narrative, and teammates affected different psychological needs rather than acting as one uniform treatment [2]. Mekler et al. reported that points, levels, and leaderboards increased output quantity in a short task but did not significantly increase intrinsic motivation [3]. These results informed a cautious interpretation: LevelUpX can record more quest completions or focus time, but those values do not prove improved task quality, intrinsic motivation, or long-term retention.

Leaderboards can also operate as implicit goals. Landers et al. found that goal commitment influenced the effect of leaderboard-based performance [4]. Therefore, the project includes both global and guild-scoped views and combines competition with cooperative team quests. The report does not claim that a ranking is suitable for every user.

2.2.2 Goal Setting and Motivation

Locke and Latham's goal-setting synthesis explains that specific and challenging goals usually perform better than vague instructions, provided that commitment, feedback, ability, and task complexity are considered [5]. A LevelUpX quest applies this through a concrete title, category, estimate, priority, due date, difficulty, project, and optional subtasks. The recovery difficulty and the ability to break work into smaller quests are especially relevant when a difficult goal would otherwise exceed current capacity.

Self-determination theory distinguishes autonomy, competence, and relatedness as important needs [6]. In the project, autonomy is supported through user-owned quests, modes, classes, focus duration, and custom rewards. Competence feedback is represented through progress, achievements, analytics, and skill paths. Relatedness is supported by

guilds, team quests, and accountability partners. These mappings are design intentions, not measured psychological outcomes. Controlling rewards or punitive streak pressure could work against autonomy. The implemented completion transaction recalculates current and longest streaks from UTC calendar-day continuity while holding the profile row lock. This provides consistent mechanics, but it remains a design feature rather than evidence of motivation or retention.

2.2.3 Focus Sessions and Break Regulation

Biwer et al. compared self-regulated breaks with systematic 24/6 and 12/3 schedules [7]. The systematic groups reported lower fatigue and distraction before breaks, but the groups did not show a significant difference in task completion. This evidence supports a configurable timer rather than a claim that one interval guarantees better learning. LevelUpX provides a 25-minute Pomodoro-style work preset, a 60-minute deep-work preset, and 5-minute short-break, and 15-minute long-break defaults. A user can also change the target, pause, resume, add a goal, save a note, or stop early.

The technical design stores a start time, pause state, accumulated pause seconds, end time, duration, completion result, optional quest, goal, and note. This gives the user a record that can be combined with quest and analytics data. However, the application does not measure attention directly, and elapsed time should not be interpreted as proof that the intended work was completed well.

2.2.4 Personal Informatics and Analytics

Li, Dey, and Forlizzi proposed a five-stage personal-informatics model: preparation, collection, integration, reflection, and action [8]. The model is relevant because a chart is only useful when the preceding records are sufficiently correct and understandable. LevelUpX reduces collection effort by linking a timer to a quest and derives weekly summaries, a 30-day heatmap, completion rates, focus consistency, and suggestions from existing records. Insight actions can open focus or create a recovery quest, connecting reflection to action.

The model also exposes current limitations. A mistaken or incomplete record can influence later analytics. The application provides edit and archive operations for projects and quests at the API level, but the current quest page does not expose every API edit operation. There is no formal study showing that users understand every chart, and no benchmark establishes that viewing the analytics improves planning decisions.

2.2.5 Workload and Burnout Terminology

The Job Demands–Resources model relates high demands mainly to exhaustion and insufficient resources mainly to disengagement [9]. This indicates that a workload total alone is not a diagnosis. The World Health Organization describes burnout as an occupational phenomenon rather than a medical condition and limits the term to chronic workplace stress [10]. LevelUpX is also used for study and personal work, so I describe the implemented rules as workload patterns, possible fatigue signals, and recovery suggestions. The software does not clinically detect or prevent burnout.

The current rules are transparent and testable. They use recent stored activity rather than a trained model. A future predictive model would require consented data, a defined target, representative samples, validation, bias review, explanations, and an appropriate health-safety boundary. These requirements are outside the present project.

2.2.6 Offline and Real-Time Foundations

The Service Workers specification defines an event-driven worker with a lifecycle and fetch interception that can support offline content [11]. In LevelUpX, the service worker pre-caches the offline fallback, manifest, and icon, and caches same-origin Next.js static assets when first fetched. Navigation remains network-first and falls back to the offline page. Structured offline quest actions are stored separately in IndexedDB. The queue distinguishes permanent client errors from transient failures and uses a client request identifier so replay does not create duplicate quests. This deliberate combination is more accurate than assuming that registration of a service worker automatically makes every feature offline.

Socket.IO preserves event order across supported transport changes, but its default delivery is at-most-once [12]. The project therefore uses Socket.IO for timely UI refreshes and notifications while durable changes remain in PostgreSQL. A user who misses a real-time event can still fetch the current state from the API. The socket server authenticates the JWT and checks GuildMember before joining a requested guild room; a non-member or membership lookup failure receives no room access. Focus-presence events similarly target only the owner and accepted accountability peers.

2.2.7 Consolidated Literature Review

Table 2.1 summarizes selected literature and design implications.

Table 2.1 Selected literature and design implications

Source	Method	Relevant finding	Limitation	Project implication
[1] Deterding et al., 2011	Conceptual definition	Gamification uses game elements in non-game contexts	Defines scope; does not prove effectiveness	XP, coins, achievements, levels
[2] Sailer et al., 2017	Randomized simulation	Different elements supported different needs	Short bundled treatment	Progress graphs, avatars, guilds
[3] Mekler et al., 2017	Controlled task experiment	Output increased without significant intrinsic-motivation change	One-time annotation task	Avoid equating activity with motivation
[4] Landers et al., 2017	Goal-setting experiment	Leaderboards can act as implicit goals	Short university task	Global/guild ranking; no opt-out
[5] Locke & Latham, 2002	Theory synthesis	Specific goals need feedback, ability, and commitment	Not a software trial	Detailed quest fields and feedback
[6] Ryan & Deci, 2000	Theory synthesis	Autonomy, competence, and relatedness support motivation	Feature effects require testing	Choice, mastery, social support
[7] Biwer et al., 2023	Student break experiment	Systematic breaks changed subjective experience, not completion	Small one-day effective sample	Configurable focus modes
[8] Li et al., 2010	Survey and interviews	Tracking moves from preparation to action	Self-selected, technologically savvy sample	Integrated analytics and actions

Source	Method	Relevant finding	Limitation	Project implication
[9] Demerouti et al., 2001	Cross-sectional job studies	Demands and resources relate differently to burnout dimensions	No causal app evidence	Cautious workload signals
[10] WHO, 2019	Term boundary	Burnout is occupational, not a medical condition	Not an assessment instrument	No diagnosis claim
[11] W3C Service Workers	Technical standard	Worker lifecycle and fetch interception enable offline patterns	No usability evidence	Offline fallback/static assets plus IndexedDB queue
[12] Socket.IO docs	Official transport docs	Ordering is distinct from delivery guarantee	No motivation evidence	Database remains authoritative

2.3 Comparison of Literature

Table 2.2 summarizes comparison of reviewed approaches.

Table 2.2 Comparison of reviewed approaches

Dimension	Literature position	LevelUpX implementation	Evidence boundary
Motivation	Game elements may affect needs and output differently	Multiple feedback forms, user choice, social and individual paths	No measured motivation or retention
Goals	Specific goals benefit from feedback and appropriate difficulty	Quest title, estimate, priority, difficulty, deadline, subtasks	Task quality is not automatically measured

Dimension	Literature position	LevelUpX implementation	Evidence boundary
Focus	Fixed breaks can alter fatigue and concentration reports	Four defaults plus configurable duration and pause/resume	No learning-outcome experiment
Analytics	Collection and integration precede reflection and action	Quest/focus records, heatmap, summaries, actionable insights	No comprehension or UX study
Workload	Demands, resources, and context matter	Rule-based signals and recovery suggestions	Not diagnosis or prevention
Offline	Service workers need deliberate data synchronization	Offline fallback/static assets, IndexedDB action, replay idempotency	Other modules require network
Real time	Event order does not guarantee arrival	Socket refreshes plus REST/database state	Guild rooms now fail closed on verified membership; delivery/reconnect evaluation remains

2.4 Official Technology Review

The implementation study also used official documentation for the selected stack. Next.js documents the App Router as a file-system router and explains pages, layouts, navigation, and client/server component boundaries [13]. React describes component-based user interfaces [14]. The TypeScript handbook explains static type checking for JavaScript programs [15]. Express documentation describes applications as ordered middleware and routing calls [16]. Prisma documents its generated type-safe client, declarative schema, and migration workflow [17]. PostgreSQL documentation was used for relational constraints and transactions [18]. Zod documentation explains schema parsing and structured validation failures [19].

Additional official sources were consulted for Socket.IO event ordering and delivery guarantees [12], Redis data structures and caching [20], Progressive Web Application behavior [21], IndexedDB [22], and WebAuthn public-key credentials [23]. These sources support technical explanations; they do not replace evidence from the project's own code and tests. Table 2.3 summarizes official technical sources consulted.

Table 2.3 Official technical sources consulted

Reference	Technology	Use in the project
[13] Next.js App Router	Next.js	File-based routes, layouts, client pages, production build
[14] React docs	React	Component and state-based user interface
[15] TypeScript Handbook	TypeScript	Strict static checking across frontend and backend
[16] Express middleware guide	Express	Middleware order, routing, request/response handling
[17] Prisma ORM docs	Prisma	Schema, generated client, migrations, PostgreSQL access
[18] PostgreSQL 16 docs	PostgreSQL	Transactions, constraints, indexes, relational storage
[19] Zod basics	Zod	Body, parameter, and query validation
[20] Redis docs	Redis	Optional cache, rate-limit store, and job locks
[21] MDN PWA	Web platform	Manifest and service-worker behavior
[22] MDN IndexedDB	Web platform	Offline structured action queue

Reference	Technology	Use in the project
[23] W3C WebAuthn	Web security	Privileged passkey registration and verification

2.5 Literature Gap and Project Position

The reviewed research studies individual mechanisms in controlled or limited settings. It does not establish that combining every mechanism will create a larger effect. LevelUpX therefore serves as an engineering project that integrates explicit goals, focus records, progression feedback, social options, analytics, offline foundations, and security controls in one implementation. Its contribution is the technical integration and verifiable workflow, not a new psychological theory.

A future evaluation should distinguish usage from outcomes. Relevant measures could include task-completion quality, planning accuracy, voluntary continued use, perceived autonomy, concentration, interruption cost, and comprehension of analytics. Such a study would require consent, a defined sample, appropriate comparison conditions, and a method approved by the institution. No such participant study was supplied for this report, so no percentage improvement or user-satisfaction claim is made.

CHAPTER III

SYSTEM REQUIREMENTS SPECIFICATION

Requirements Approach

I derived the requirements from the implemented user journeys, route definitions, validation schemas, Prisma data model, test cases, environment contract, and project notes. This is therefore an implementation-backed requirements specification rather than a record of stakeholder interviews. Where a requirement depends on a production provider, physical device, or formal user study that was not available, I state the dependency explicitly.

I use the word shall for a required externally observable behavior, should for a quality target, and may for an optional configuration. Each functional requirement has a stable identifier so that the design, implementation, test evidence, and result can be traced. The identifiers describe the current local release and do not imply that deferred production operations have been completed.

3.1 Functional Requirements

Table 3.1 summarizes functional requirements.

Table 3.1 Functional requirements

ID	Area	Requirement
FR-01	Identity	The system shall register a user only after validating the name, normalized email address, and password.
FR-02	Identity	The system shall withhold an authenticated session until the emailed verification challenge is completed.
FR-03	Identity	The system shall support login, refresh, logout, password recovery, session review, and session revocation.
FR-04	Identity	The system shall enforce role checks for USER, ADMIN, and SUPER_ADMIN operations.
FR-05	Privileged access	The system shall apply trusted-device, multiple-code, and passkey stages to privileged authentication.

ID	Area	Requirement
FR-06	Onboarding	The system shall collect an initial mode, class, preferences, and first quest before completing onboarding.
FR-07	Projects	A user shall create, list, update, and archive owned projects.
FR-08	Quests	A user shall create a quest with validated planning fields and optional project association.
FR-09	Quests	A user shall list and filter quests, view one quest, change lifecycle state, edit it, and retire an eligible item.
FR-10	Quests	The system shall prevent duplicate quest-completion payout and record a completion transaction atomically.
FR-11	Offline	The browser shall queue supported quick-capture quest creation without storing the access token in IndexedDB.
FR-12	Offline	The client shall replay a queued capture with a stable client request identifier and remove it only after success.
FR-13	Focus	A user shall start work or break sessions with a selected or custom duration and optional quest link.
FR-14	Focus	A user shall pause, resume, stop with a completion result, annotate, and review focus records and statistics.
FR-15	Progression	The system shall calculate stored quest rewards with one difficulty adjustment, then apply streak/economy multipliers, coin inflation, caps, and the remaining daily coin limit at completion.
FR-16	Progression	The system shall calculate reward, UTC streak, and daily coin allowance and maintain XP/coin histories, balance, and level inside the same-user locked completion transaction.
FR-17	Progression	The system shall evaluate achievements and prerequisite-gated skill progression without awarding an owned achievement or advancing a locked skill.

ID	Area	Requirement
FR-18	Rewards	A user shall browse, purchase, and equip supported cosmetic items and use supported marketplace operations subject to eligibility and balance.
FR-19	Rewards	A user shall define a personal reward and record redemption subject to its coin cost.
FR-20	Social	A user shall create, join, leave, and inspect guilds under membership and ownership rules.
FR-21	Social	Authorized guild members shall create or contribute to team quests within their active date window, optionally recur them weekly without duplicate or missed historical materialization, and receive a controlled completion payout.
FR-22	Social	The system shall provide global or guild-scoped leaderboard views from authoritative stored data.
FR-23	Accountability	A user shall request, accept, decline, list, block, and remove an accountability connection; focus presence shall be visible only to the owner and accepted peers, and block/remove shall clear it to IDLE.
FR-24	Analytics	A user shall view quest, focus, reward, and activity summaries for supported periods.
FR-25	Insights	The system shall produce clearly labelled rule-based productivity and workload guidance from stored records.
FR-26	Notifications	A user shall list and mark notifications and manage notification and push-subscription preferences.
FR-27	Safety	A user shall submit/review owned abuse reports, while rapid or repetitive completion rules shall create concurrency-safe deduplicated review signals without rolling back an already committed quest reward.
FR-28	Account	A user shall export personal data and permanently delete the account after required confirmation.

ID	Area	Requirement
FR-29	Administration	An authorized administrator shall use a dedicated login entry, inspect users/reports/health/metrics/events, and export supported analytics and marketplace records as CSV or PDF.
FR-30	Administration	A super administrator shall manage supported role changes and reserved system health actions; verified administrators shall manage their own passkeys and trusted devices under policy.

3.2 Non-Functional Requirements

Table 3.2 summarizes non-functional requirements.

Table 3.2 Non-functional requirements

ID	Quality	Requirement or target
NFR-01	Security	Passwords shall be one-way hashed; access and refresh credentials shall have distinct handling and expiry.
NFR-02	Security	Protected routes shall reject missing, invalid, expired, or insufficiently privileged credentials.
NFR-03	Security	Unexpected server errors shall not reveal internal database or stack details to the caller.
NFR-04	Security	Validation and rate limiting shall be applied before high-risk business operations.
NFR-05	Integrity	Quest payout and the corresponding state and ledger updates shall share an atomic database transaction.
NFR-06	Integrity	Retryable client creation shall use idempotency so one logical capture creates no more than one quest.
NFR-07	Reliability	The PostgreSQL database shall remain authoritative when real-time events are delayed or lost.
NFR-08	Maintainability	Frontend and backend source shall pass strict TypeScript checks and zero-warning lint commands.

ID	Quality	Requirement or target
NFR-09	Maintainability	Backend business areas should preserve route, validation, controller, service, and repository boundaries.
NFR-10	Testability	Business rules, HTTP boundaries, relational transactions, and critical browser journeys should have separate tests.
NFR-11	Usability	Primary user workflows shall be reachable from consistent grouped navigation and provide visible loading/error states.
NFR-12	Accessibility	Controls should use semantic elements, keyboard-operable interactions, labels, focus indicators, and readable contrast.
NFR-13	Responsiveness	Main screens should remain usable on desktop and narrow mobile-width viewports without horizontal page overflow.
NFR-14	Performance	The client should paginate or bound large histories, and frequently read ranked data may use a short-lived cache.
NFR-15	Observability	The API shall expose structured logs, health information, and guarded metrics when configured.
NFR-16	Privacy	Offline storage shall exclude bearer credentials, and export/delete operations shall require an authenticated owner.
NFR-17	Portability	The application should run through documented Node.js commands or the supplied local container topology.
NFR-18	Recoverability	A production installation shall define automated backups and restore tests before real user data is accepted.
NFR-19	Compatibility	The current supported browser evidence is Chromium; broader browser support requires a future verification matrix.

ID	Quality	Requirement or target
NFR-20	Ethics	Workload insights shall not be described as diagnosis, medical advice, or proof of burnout prevention.

3.3 Hardware Requirements

The software does not depend on specialist hardware for ordinary use. A modern browser and network connection are sufficient for user workflows, while local development needs enough memory and disk space for two Node.js applications and the optional container stack. The values below are planning estimates only; I did not establish them through resource benchmarks or production capacity tests. Table 3.3 summarizes hardware requirements.

Table 3.3 Hardware requirements

Context	Minimum practical configuration	Recommended configuration
End user	Dual-core device, 4 GB RAM, 1366 x 768 display or modern phone, browser storage	Recent desktop or phone, 8 GB RAM, reliable network
Local application	64-bit processor, 8 GB RAM, 10 GB free disk	Four or more cores, 16 GB RAM, SSD, 20 GB free disk
Container stack	Hardware virtualization, 12 GB available RAM	16 GB or more RAM and SSD for all monitoring services
Privileged login	Platform authenticator or compatible WebAuthn credential	Two registered trusted devices and a recovery procedure
Academic demo target	Up to 25 concurrent users	Planning target only; the application has not been load-tested

3.4 Software Requirements

Table 3.4 summarizes software requirements.

Table 3.4 Software requirements

Layer	Selected software	Purpose
Operating system	Windows, Linux, or macOS development host	Runs Node.js, package scripts, and optional containers
Runtime	Node.js 22.16.0 observed; compatible current Node release	Executes Next.js, Express, build tools, and scheduler
Package manager	npm 11.7.0 observed	Installs locked dependencies and runs project scripts
Frontend	Next.js 16.3.0, React 19.2.8, TypeScript, Tailwind CSS	Responsive routes, components, state, and production bundle
Backend	Express, TypeScript, Zod, Prisma Client 5.22.0	HTTP routes, validation, business rules, and data access
Data	PostgreSQL 16; optional Redis	Durable relational state; optional cache, rate limits, and locks
Realtime/jobs	Socket.IO and scheduler process	Live refresh events and recurring background tasks
Testing	Vitest, Supertest, jsdom, Playwright 1.62.1	Unit, HTTP, database, UI-unit, and browser verification
Local delivery	Docker Engine/Desktop with Compose	Reproducible multi-service demonstration topology
Browser	Current Chromium-family browser	PWA, IndexedDB, service worker, and WebAuthn-compatible UI

3.5 User Roles and Privileges

Table 3.5 summarizes user roles and privileges.

Table 3.5 User roles and privileges

Role	Permitted capability	Key restriction
Visitor	Open landing, registration, login, verification, and recovery screens	Cannot access protected application data
USER	Own productivity, progression, social, notification, insight, and account workflows	Cannot use administrator route families
ADMIN	Authorized moderation and operational views after privileged verification	Cannot perform reserved super-administrator policy actions
SUPER_ADMIN	Reserved role changes and system-health actions plus verified-admin control-room capabilities	Cannot bypass staged authentication or protected self-action rules
Scheduler	Runs configured background jobs using internal application services	Not an interactive user and must not expose a public login

3.6 Data Requirements

The data model must preserve ownership, referential integrity, uniqueness, lifecycle timestamps, and transaction history. UserProfile stores calculated-facing progression state, while XpTransaction and CoinTransaction retain the reason and amount of each change. QuestCompletion is separate from Quest so a completed work event can be audited. Relationships connect a user to owned projects, quests, focus sessions, achievements, skills, cosmetics, guild membership, sessions, reports, preferences, and events.

Important uniqueness requirements include normalized account email, one profile per user, one unique completion per quest, one membership per user and guild, and stable client request identifiers where replay protection is required. Mutable records use created and updated timestamps. Security-sensitive records also include expiry, consumption,

revocation, or last-used state. The Prisma schema is the exact model-by-model authority and the principal entities are summarized in Chapter IV.

3.7 Interface Requirements

- User interface: server-delivered web routes with client-side interactive components, responsive navigation, forms, charts, dialogs, tables, and status messages.
- HTTP interface: JSON REST endpoints below `/api` with bearer access authentication where required and a consistent structured error body.
- Realtime interface: authenticated Socket.IO connection for selected notification, guild, and leaderboard refresh events; REST remains authoritative.
- Persistence interface: Prisma Client access to PostgreSQL through repository functions and transactions.
- Email interface: configured SMTP provider for verification, one-time login, recovery, security notification, and digest messages; Mailpit is used locally.
- Push interface: optional Web Push subscription storage and delivery credentials; production provider values remain external inputs.
- Observability interface: structured application logs, health endpoints, protected metrics, and optional Sentry-compatible error reporting hooks.

3.8 Assumptions, Constraints, and Acceptance

The current build assumes a correctly migrated PostgreSQL database, compatible environment variables, and system clocks accurate enough for token and challenge expiry. Redis is optional for supported fallbacks, but PostgreSQL is not optional. Email-dependent journeys need an SMTP provider. Privileged passkeys need a secure context and a compatible authenticator. Offline replay assumes that the user later reconnects and establishes a valid in-memory session.

The project is constrained by an individual academic schedule and local infrastructure. Acceptance is based on source-backed behavior, successful available verification commands, and traceability to the stated objectives. A public release additionally requires domain, TLS, secrets, backups, restore tests, production messaging providers, monitored service-level targets, and a security review. These items are outside the evidence available for the current report.

1. Both packages include lockfiles, and the installed dependency sets pass strict type checking and production builds.
2. The frontend and backend create production builds without an error.
3. Available unit and HTTP integration suites pass in the verification environment.
4. Database and critical browser workflows pass in a correctly provisioned isolated environment before public release.
5. No deferred provider or production-control claim is treated as complete without direct evidence.

CHAPTER IV

SYSTEM DESIGN

4.1 Design Overview

I selected a modular-monolith design because it gives one deployable API while preserving explicit business boundaries. The browser communicates with the REST API and an authenticated real-time channel. The API coordinates validation, authentication, business services, repositories, PostgreSQL, optional Redis facilities, email or push providers, and metrics. A scheduler imports the same domain services for recurring work instead of duplicating business rules.

The separation is logical rather than a claim of independent microservices. A module can be tested in isolation, but modules share one runtime, one Prisma schema, and one database. This reduces deployment overhead for a student project and makes cross-module transactions practical. It also means that future scaling should begin with measurement rather than premature service extraction. Figure 4.1 illustrates high-level system architecture.

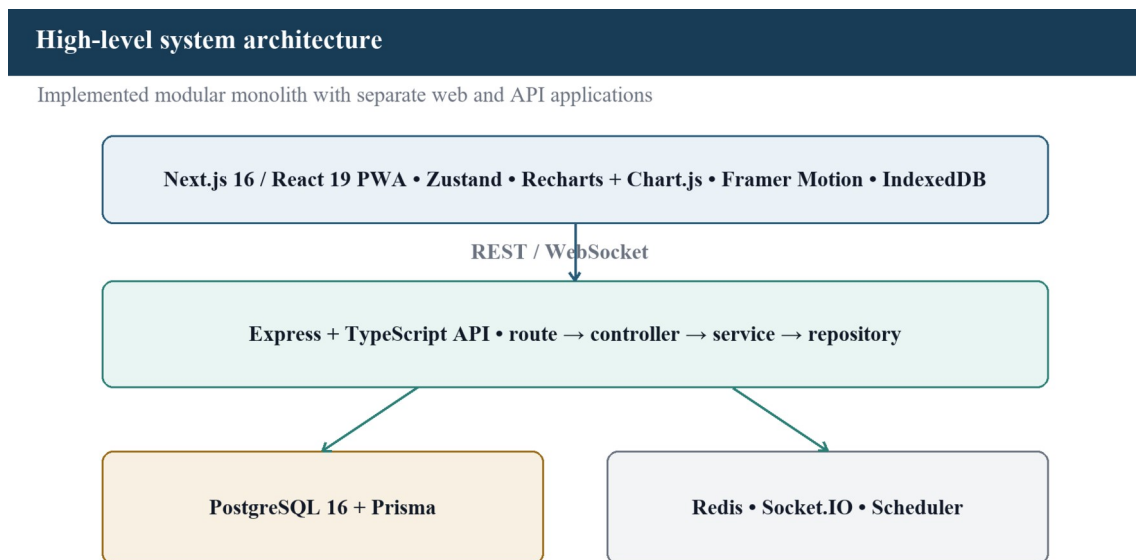


Figure 4.1 High-level system architecture

4.2 System Architecture

4.2.1 Frontend Layer

The Next.js App Router maps files to public, authentication, onboarding, user, and administration pages. React components render forms, cards, tables, charts, dialogs, navigation, timer controls, and feedback. A small authentication store keeps the short-lived access token in memory rather than browser local storage. API utilities attach credentials, attempt controlled refresh, normalize errors, and return typed data. IndexedDB is reserved for the supported offline action queue.

4.2.2 Backend Layer

Express composes security headers, origin policy, request parsing, request identifiers, logging, route rate limits, authentication, authorization, module routers, not-found handling, and a centralized error response. Most domain modules contain a validation file, controller, service, repository, routes, and tests. Services enforce rules; repositories issue Prisma operations; controllers translate HTTP data.

4.2.3 Data, Realtime, and Job Layers

PostgreSQL stores durable identity, productivity, progression, social, safety, notification, and operational records. Redis can supply a distributed rate-limit store, short-lived cache, and scheduler locks, while the implementation retains controlled fallbacks for supported local cases. Socket.IO rooms target a user or guild for refresh events. Because transport arrival is not guaranteed, the client re-reads authoritative API state after important signals. Recurring jobs prepare reports, refresh ranked views, send reminders and digests, and generate rule-based insight records. Figure 4.2 illustrates functional module map.

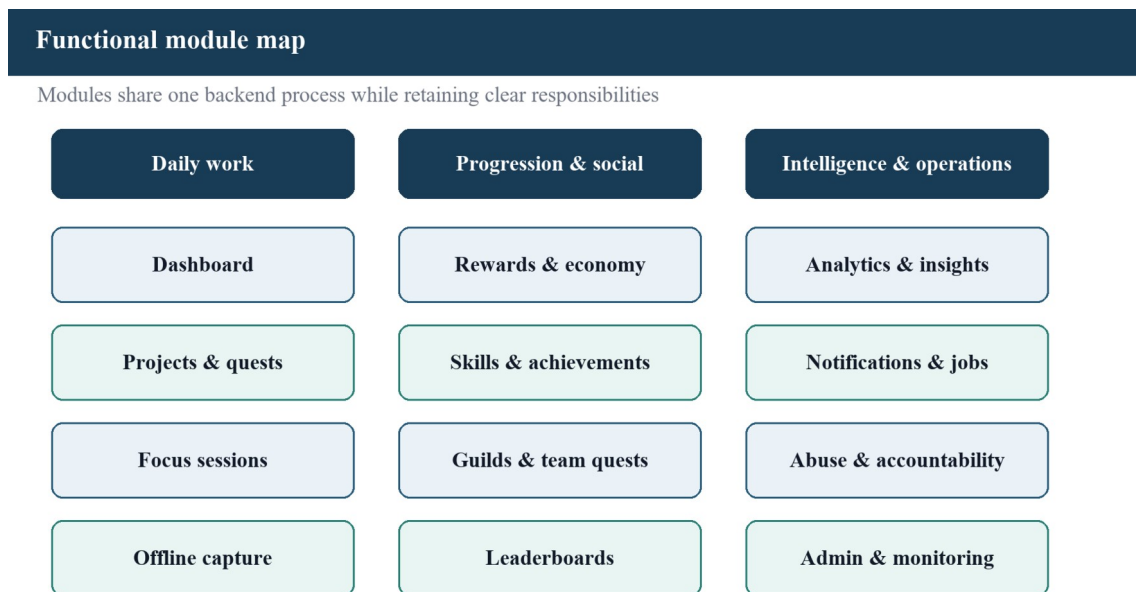


Figure 4.2 Functional module map

Table 4.1 summarizes backend module responsibilities.

Table 4.1 Backend module responsibilities

Module	Primary responsibility	Representative data
auth	Registration, verification, login, recovery, refresh, sessions, privileged challenges	User, UserSession, trusted device and challenge records
account	Profile export and permanent account deletion	User-owned aggregate
projects	Owned project lifecycle	Project

Module	Primary responsibility	Representative data
quests	Quest planning, lifecycle, recurrence, idempotent completion	Quest, QuestCompletion
focus	Work and break session lifecycle and history	FocusSession
rewards	XP/coin summary, history, cosmetics, custom rewards	Ledgers, cosmetics, redemptions
achievements	Eligibility evaluation and earned achievements	Achievement, UserAchievement
users/skills	Profile and skill progression	UserProfile, Skill, UserSkill
guilds	Membership, invitation, team quests, contributions	Guild, GuildMember, TeamQuest
leaderboard	Global and scoped ranking views	Leaderboard and derived profile totals
accountability	Connection request and relationship lifecycle	AccountabilityConnection
analytics	Activity, completion, focus, and reward summaries	Read models from user history
aiInsights	Rule-based or provider-assisted insight contract	AiInsight, AiPromptRun
notifications	Inbox, preferences, subscriptions, delivery coordination	Notification, preferences, push subscriptions
abuse	User reporting and owned status review	AbuseReport
admin	Control room, moderation, roles, economy, reports, telemetry	AdminAction, AdminReport, EconomySettings
productEvents	Consent-aware product funnel events	ProductEvent

Module	Primary responsibility	Representative data
health	Health and protected metrics endpoints	Runtime and configured dependency status

4.3 Data Flow Diagrams

At the context level, the user and privileged administrator are the interactive external entities. SMTP, optional push delivery, and observability systems are supporting external services. The single LevelUpX system boundary contains the web client, API, scheduler, and owned data stores. Credentials, quest commands, focus records, social actions, report decisions, and configuration enter the boundary; views, confirmations, notifications, exports, and metrics leave it. Figure 4.3 illustrates context-level data flow diagram.

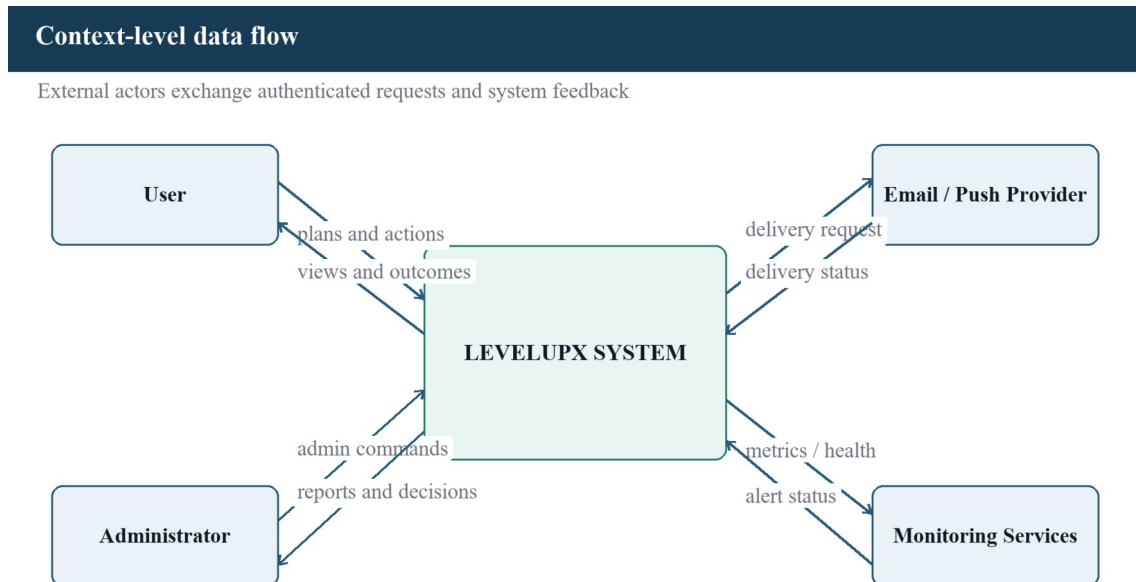


Figure 4.3 Context-level data flow diagram

The level-1 view separates identity, productivity, progression, social, insight, notification, and administration processes. Identity produces an authenticated actor. Productivity stores quests and focus sessions. Progression consumes verified actions and writes ledgers. Social processes enforce membership around guild data. Insight and analytics read the user's own history. Administration reads operational data and writes auditable decisions. The separation clarifies which process owns a rule even though the implementation shares one API runtime. Figure 4.4 illustrates level-1 data flow diagram.

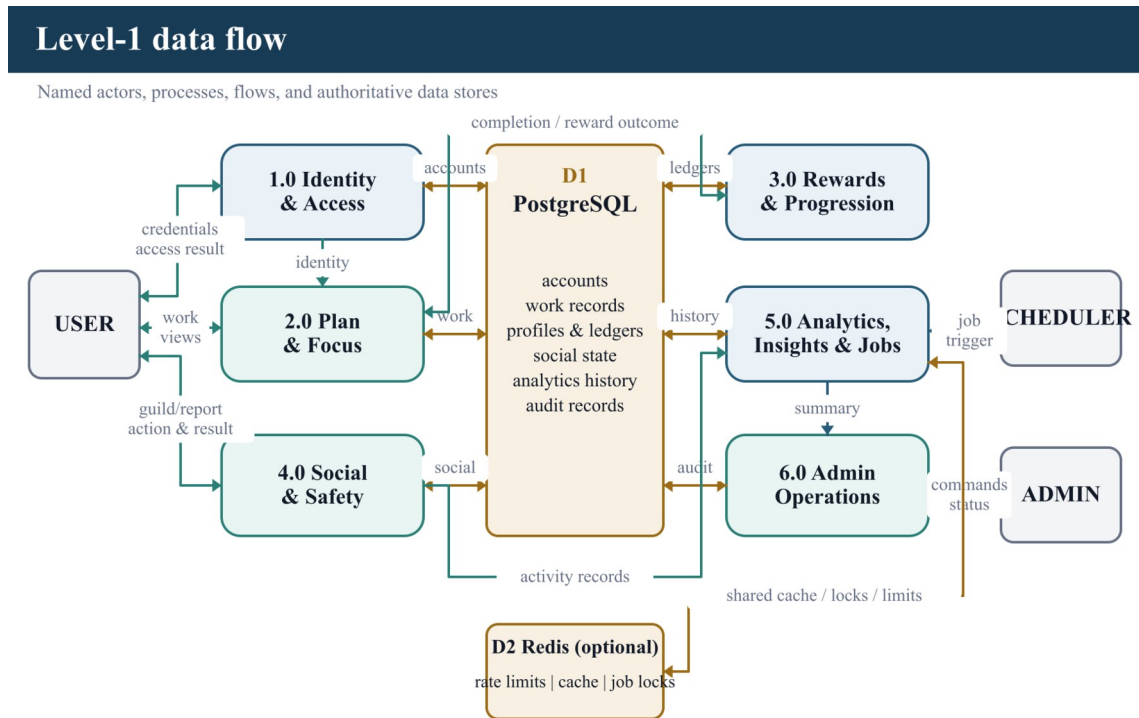


Figure 4.4 Level-1 data flow diagram

4.4 Use Case Diagram

Figure 4.5 illustrates use-case overview.

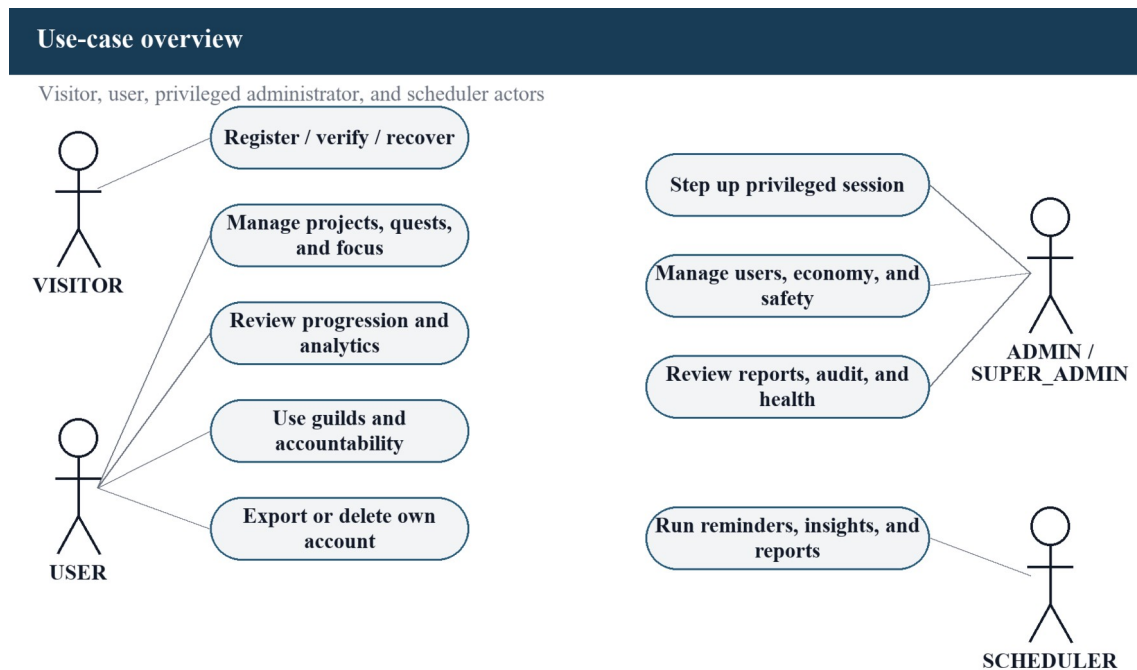


Figure 4.5 Use-case overview

The primary actor is the authenticated user. The user's central use case is to plan and complete work, supported by focus, progress, analytics, social accountability, notification, and account-control use cases. A visitor can register, verify, log in, or recover access. An administrator inherits only the ordinary authenticated foundation and must satisfy the privileged stages before using the control room. The scheduler is a supporting actor that initiates time-based jobs without an interactive browser.

4.4.1 Main Success Scenario: Complete a Quest

1. The authenticated user selects an owned quest that is in a completable state.
2. The client sends the completion request and the API verifies identity and input.
3. The quest service loads the applicable economy settings and user progression state.
4. The reward rule calculates XP and coins under difficulty multipliers and configured caps.
5. A database transaction changes quest state, creates one completion, writes ledger rows, and updates profile totals.

6. Achievement evaluation runs against the resulting state and records only newly earned achievements.
7. The API returns the authoritative result and selected realtime notifications request affected views to refresh.

4.4.2 Alternate and Failure Scenarios

- If the quest is not owned by the actor, the request is rejected without revealing another user's data.
- If the state is not completable or a payout already exists, the service returns a conflict and creates no second reward.
- If any database update fails, the transaction rolls back the state transition and all associated balances and ledgers.
- If a realtime event is lost, a later REST read still returns the durable state.

4.5 Class Diagram

Figure 4.6 illustrates UML class and dependency diagram.

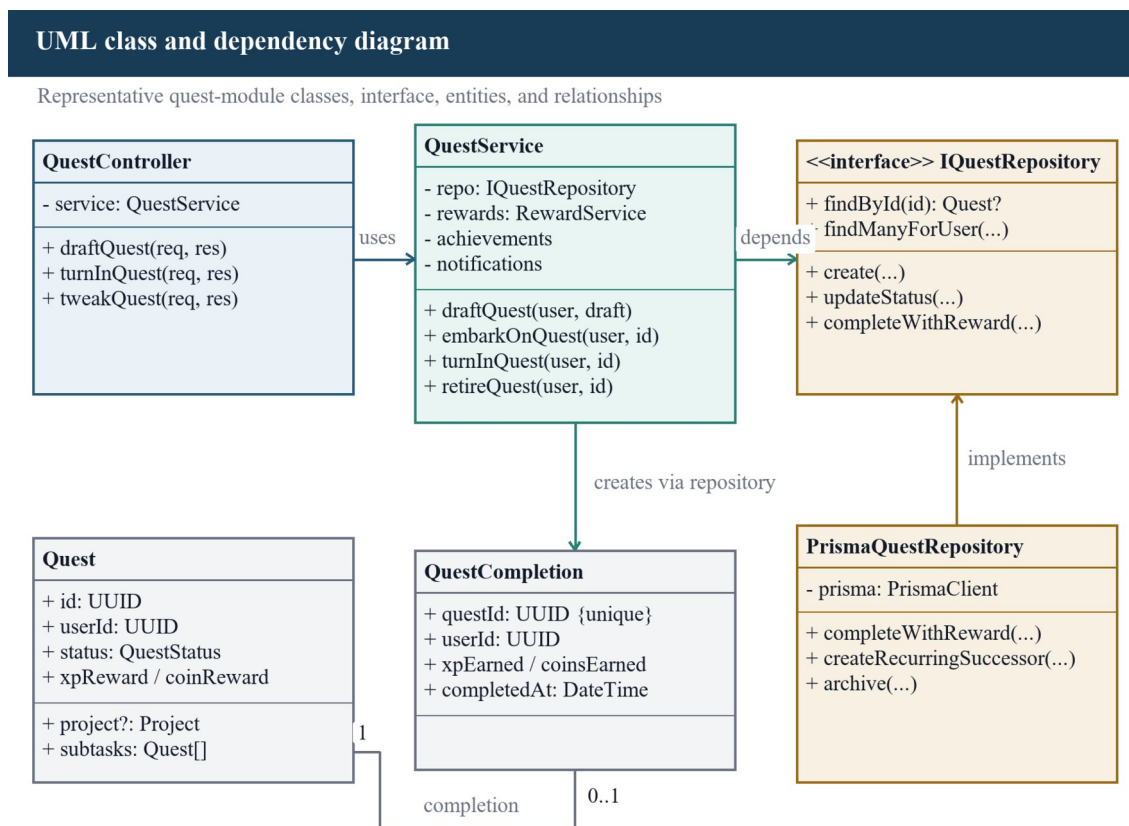


Figure 4.6 UML class and dependency diagram

The dependency direction runs from routes to controllers, controllers to services, and services to repositories or provider interfaces. Validation occurs before business methods receive untrusted input. Services are tested with repository and provider doubles so a rule can be exercised without a database. Database tests then verify transaction and concurrency behavior using the real Prisma adapter. This is a layered dependency view; TypeScript functions and objects are used where a traditional class hierarchy would add no benefit.

4.6 Sequence Diagram

4.6.1 Privileged Authentication

Figure 4.7 illustrates privileged authentication sequence.

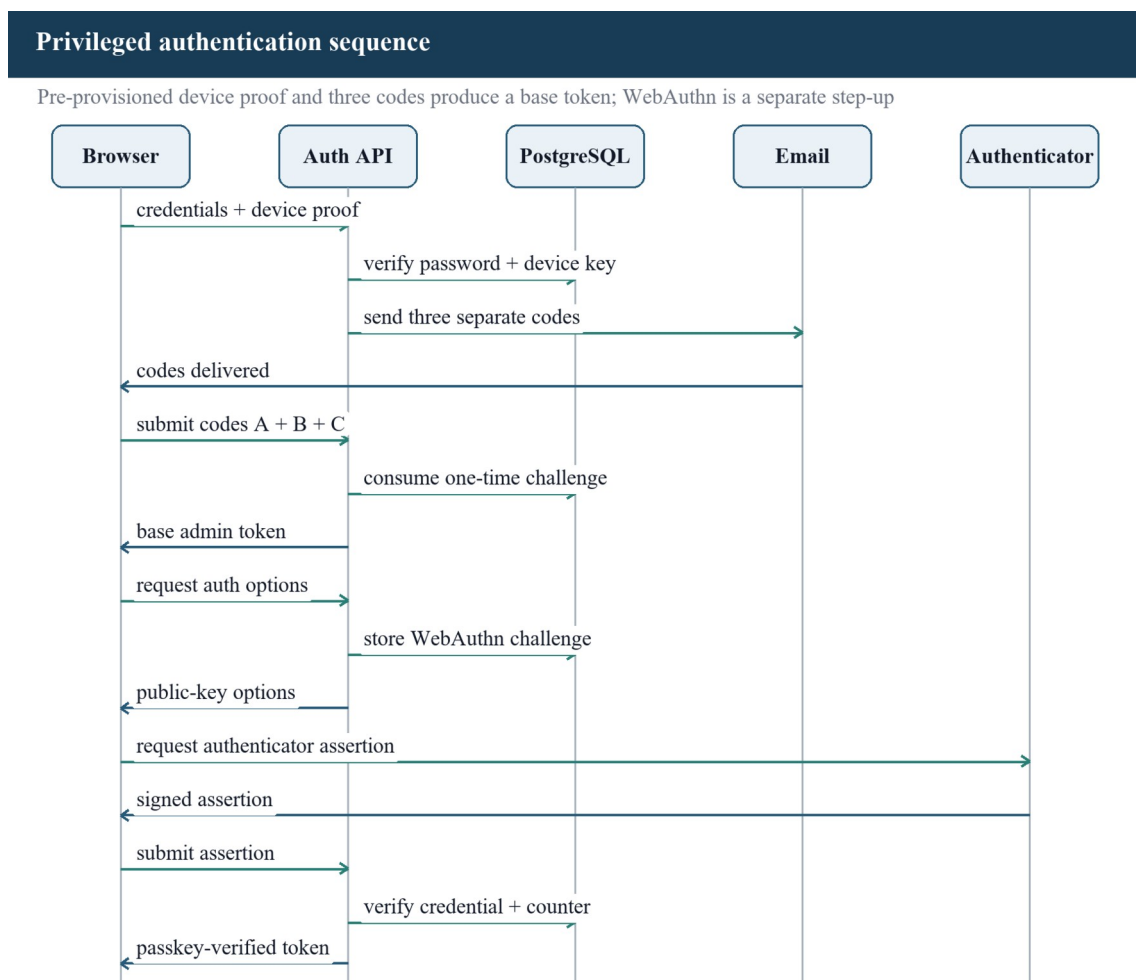


Figure 4.7 Privileged authentication sequence

Privileged authentication is intentionally staged. Password verification alone does not produce a control-room session. The flow proves a pre-provisioned trusted-device key, consumes three short-lived code challenges under concurrency controls, issues a base

admin token, and then verifies a WebAuthn credential to issue the passkey-verified token accepted by privileged routes. Middleware later checks that the trusted device remains active so revocation can invalidate an otherwise valid token. Exact production relying-party identifiers and origins must match the deployed HTTPS domain.

4.6.2 Quest Completion

Figure 4.8 illustrates quest completion sequence.

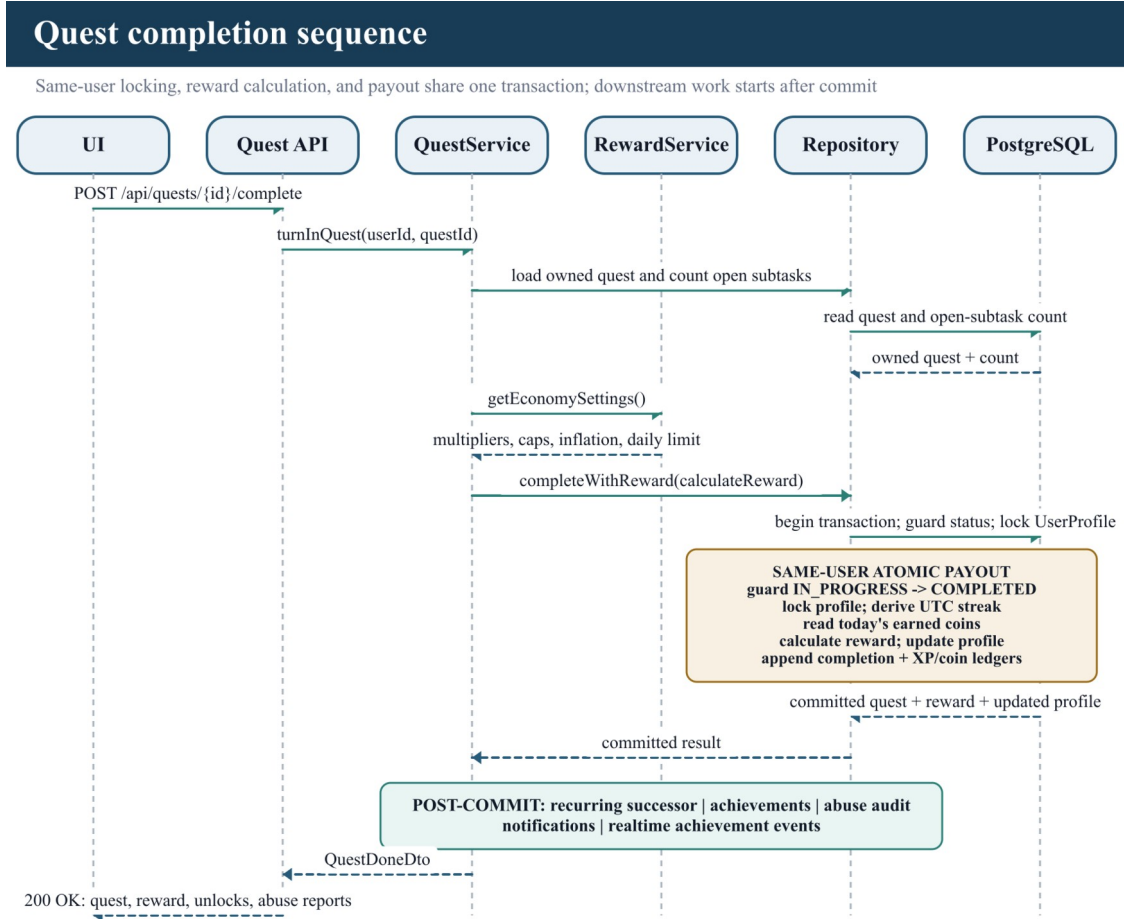


Figure 4.8 Quest completion sequence

The quest-completion sequence places the state transition, same-user profile lock, UTC streak projection, remaining daily coin calculation, reward calculation, completion record, XP and coin ledgers, and profile update inside one database transaction. The guarded state change and unique completion rule defend against duplicate payout, while the profile lock serializes concurrent completions for one user. Subsequent achievement and notification work is based on the committed result. The reward arithmetic also has isolated tests; its database-backed concurrency cases are collected but skipped when no isolated test database is configured.

4.7 Activity Diagram

Figure 4.9 illustrates quest lifecycle activity diagram.

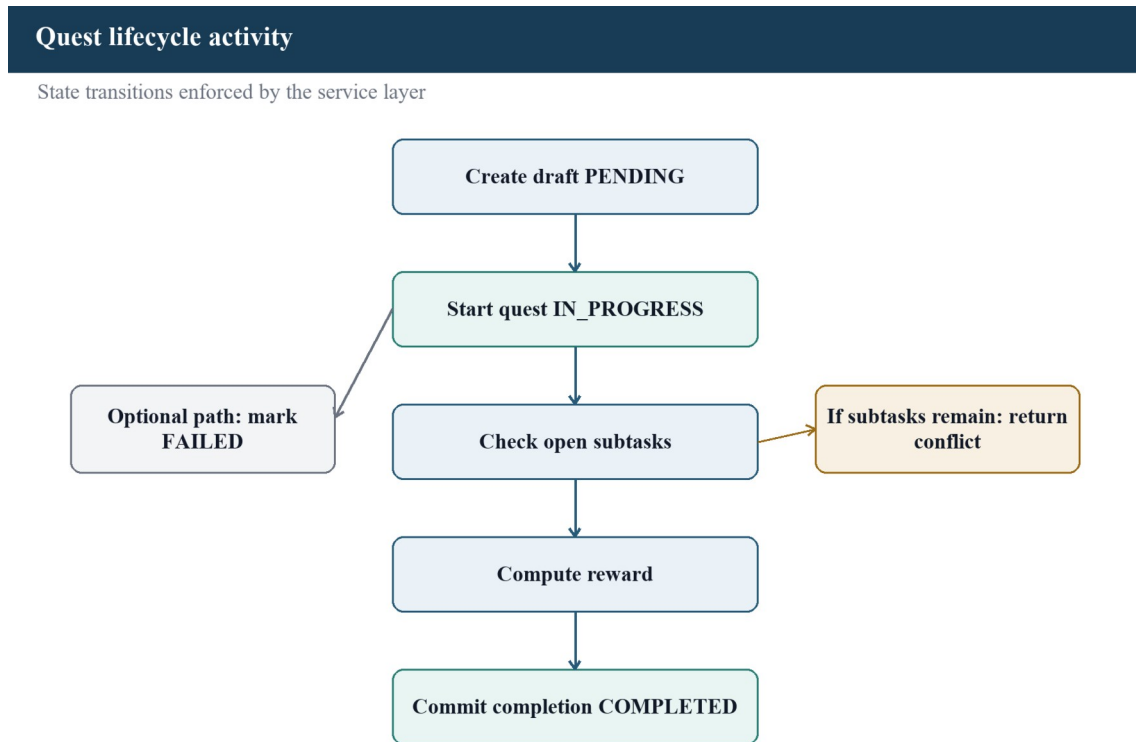


Figure 4.9 Quest lifecycle activity diagram

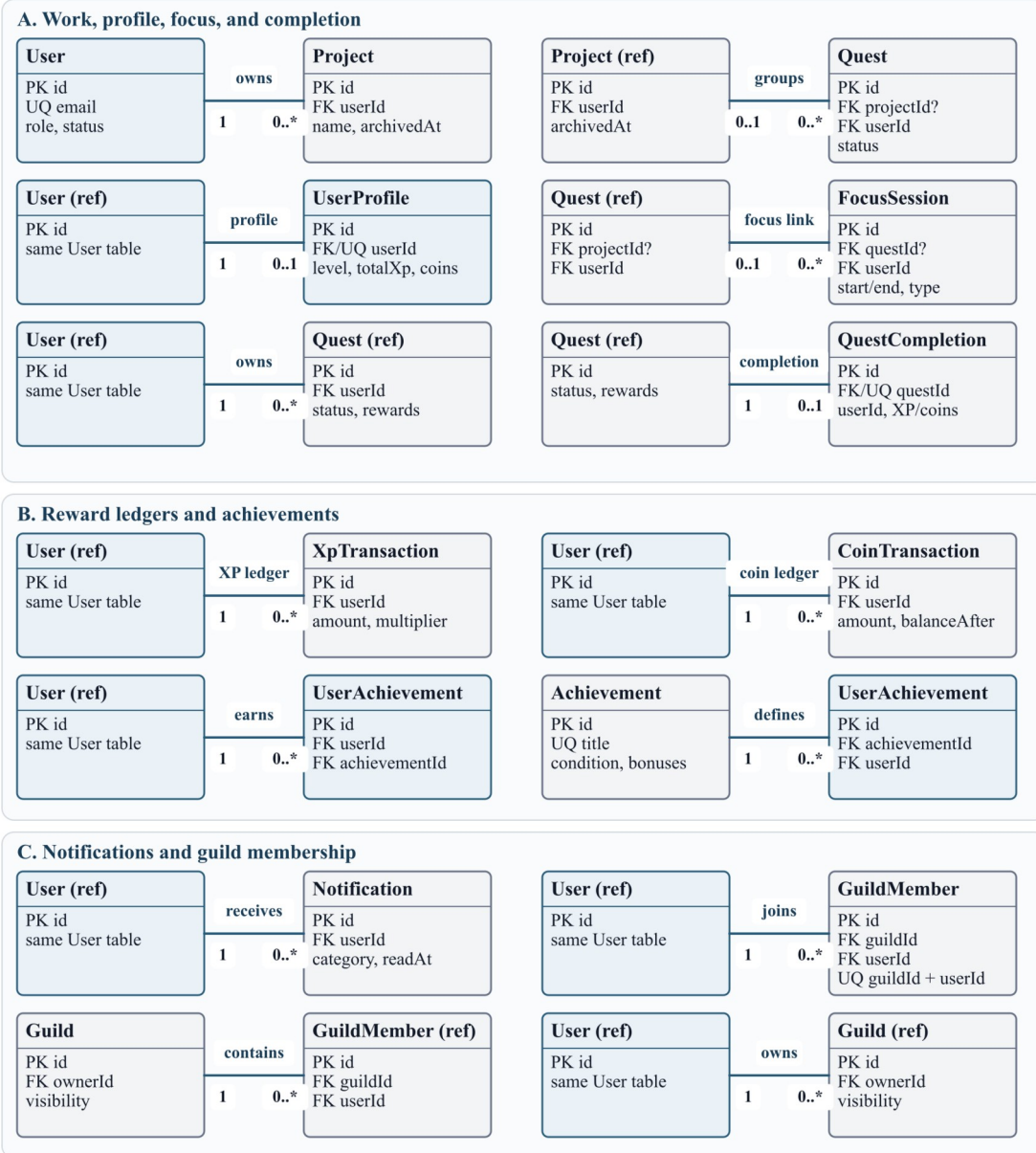
A quest is created in PENDING, may be edited while eligible, and can move to IN_PROGRESS through the start action. It becomes COMPLETED only through guarded completion, FAILED through the fail action, or ARCHIVED through the delete/archive action. A focus session is related but does not by itself authorize a reward. This prevents timer manipulation from being treated as completed work. Recurrence creates or schedules another logical quest rather than reopening and paying the same completed record indefinitely.

4.8 E-R Diagram

Figure 4.10 illustrates core entity-relationship diagram.

Core entity-relationship diagram

Representative keys and schema cardinalities; '(ref)' boxes repeat the same Prisma model



Nullable quest links produce 0..1 Project and 0..1 Quest cardinalities; a Guild may have 0..* stored members.

Figure 4.10 Core entity-relationship diagram

The Prisma schema contains 39 models. The diagram shows the core rather than every operational table so that ownership and reward relationships remain legible. User is the account root; UserProfile holds the current presentation state; ledger and completion tables are append-only through the current application workflows and retain the evidence behind that state. Project, Quest, and FocusSession represent planned and performed work. Join entities represent achievements, skills, cosmetics, guild membership, contributions, subscriptions, and accountability relationships. Table 4.2 summarizes core database entities.

Table 4.2 Core database entities

Entity group	Models	Design purpose
Identity/ security	User, UserSession, AdminPasskey, AdminWebAuthnChallenge, SuperAdminTrustedDevice, SuperAdminLoginChallenge	Account, refresh-session, trusted-device, code, and credential lifecycle
Profile/ progression	UserProfile, CharacterClass, Skill, UserSkill, Achievement, UserAchievement	Current identity, level, class, skill, and earned-achievement state
Productivity	Project, Quest, QuestCompletion, FocusSession	Planning hierarchy, work lifecycle, one-per-quest completion evidence, timed sessions
Economy	XpTransaction, CoinTransaction, EconomySettings	Auditable value changes and configurable calculation bounds
Rewards	CosmeticItem, UserCosmetic, MarketplaceListing, CustomReward, CustomRewardRedemption	Virtual items, ownership, listing, and personal redemption
Social	Guild, GuildMember, TeamQuest, TeamQuestContribution, Leaderboard, AccountabilityConnection	Membership, team progress, ranking, and peer relationships

Entity group	Models	Design purpose
Insight/ communication	AiInsight, AiPromptRun, Notification, UserNotificationPreference, UserPushSubscription	Generated guidance, prompt audit, inbox, and delivery preferences
Safety/ operations	AdminAction, AdminReport, AbuseReport, ProductEvent	Audit trail, scheduled/administrative reports, moderation, and product telemetry

4.9 Modules of the Project

The functional module map in Figure 4.2 and the responsibility matrix in Table 4.1 identify the implemented project modules and their principal data. Together, the modules cover identity and account control, projects and quests, focus, progression and rewards, social collaboration, analytics and insights, notifications and safety, administration, product events, health, and scheduled operations. Their service boundaries remain logical parts of one modular monolith and share the relational model.

I described the modules by their real inputs, processing, stored records, outputs, and current limits because a route count alone does not explain how the system works. Identity and account control establish who may act. Productivity modules convert planning and focus input into durable work records. Progression modules derive bounded feedback from completed work. Social modules apply membership or accepted-peer authorization before sharing state. Analytics and communication modules read authoritative records and present summaries or delivery requests. Administrative and operational modules apply stronger access checks because their actions affect other users or the running system. Table 4.3 summarizes module inputs, processing, outputs, and boundaries.

Table 4.3 Module inputs, processing, outputs, and boundaries

Module group	Inputs and processing	Outputs and current boundary
Identity and account	Name, email, password, verification/recovery codes, session or deletion command. Normalize identity, hash secrets, rotate one opaque refresh-session row atomically, and enforce role and ownership.	Produces verified sessions, revocation state, personal export, or deletion result. Hosted email and physical passkeys are unverified; family-wide refresh replay revocation is not implemented.
Projects, quests, and offline capture	Planning fields, lifecycle commands, optional project link, and a client request ID. Validate ownership/state, persist planning data, and replay one quick-capture operation idempotently.	Produces owned project and quest views with authoritative lifecycle state. Offline mutation is limited to client-assisted quick quest capture; some API edits need broader UI/browser coverage.
Focus and progression	Work/break duration, optional quest, session commands, and eligible quest completion. Maintain focus state and calculate UTC streak, daily allowance, reward, and ledgers under the locked completion transaction.	Produces focus history/statistics, XP, coins, level, streak, achievements, and skill state. There is no automatic Pomodoro cycle or attention-quality measurement; the relational lane must be rerun.

Module group	Inputs and processing	Outputs and current boundary
Rewards and social	Cosmetic, custom-reward, guild, contribution, and accountability commands. Check balance, ownership, membership, date window, and accepted relationship before updating records or authorizing realtime scope.	Produces inventory/marketplace state, guild/team progress, rankings, and accepted-peer presence. Items have no cash value; broader browser, reconnection, ranking-privacy, and load evidence is pending.
Analytics, insights, and notifications	Stored activity windows, preferences, feedback, and provider configuration. Aggregate activity, apply local rules or an optional provider contract, clamp scores, and persist insight, prompt-run, and notification records.	Produces charts, heatmaps, personal CSV, administrative CSV/PDF, bounded guidance, inbox, and delivery requests. Optional external text is not fully semantic-filtered; hosted delivery and participant comprehension are unverified.
Admin, safety, and operations	Privileged sessions, moderator/economy commands, job triggers, and health/metrics requests. Recheck role and device, record audit/export metadata, run scheduled services, and expose guarded telemetry.	Produces moderation/economy results, OPEN review signals, audit evidence, exports, job results, health, and metrics. Deployment hooks, live readiness, provider operations, recovery, capacity, and penetration evidence remain deferred.

The table also shows the dependency direction used during implementation. Routes and controllers collect input, services make the business decision, and repositories or providers perform persistence or external delivery. PostgreSQL remains authoritative when cache entries or realtime messages are unavailable. This separation made it possible to test a service with controlled doubles while retaining dedicated HTTP and database lanes for the boundaries that mocks cannot establish.

4.10 Offline and Synchronization Design

Figure 4.11 illustrates offline quest capture and replay flow.

Offline quest capture and replay

Only quick-capture quest creation is queued offline

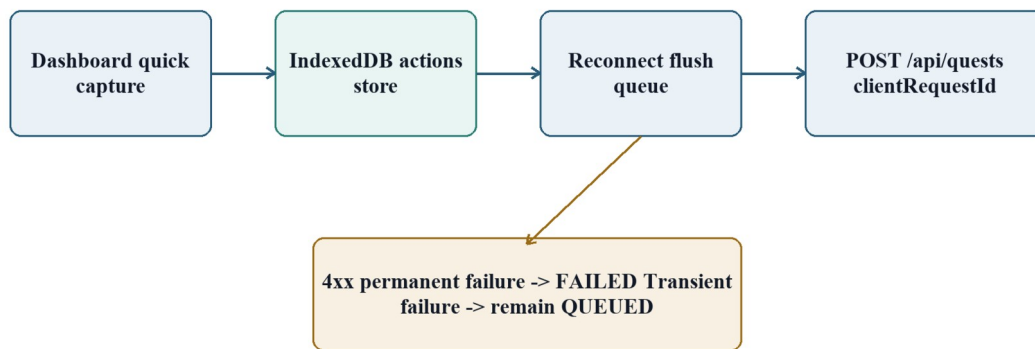


Figure 4.11 Offline quest capture and replay flow

The service worker supports application-shell behavior, while IndexedDB stores only the supported quick quest command and non-secret payload. The queue excludes the bearer token. After the connection and session are restored, replay adds a stable client request identifier. The backend treats that identifier as an idempotency key so a browser retry or uncertain response does not create two quests. Invalid or permanently rejected actions remain visible for user resolution rather than being silently discarded.

This is not a general conflict-resolution engine. Editing quests, completing work, spending coins, and social operations require current server state. Limiting offline mutation to one bounded workflow made the replay rule testable and avoided unsafe reconciliation of balances or membership.

4.11 API and Error Design

Table 4.4 summarizes API response and error conventions.

Table 4.4 API response and error conventions

Concern	Design	Reason
Base path	Module routers are mounted below /api	Separates application endpoints from frontend routes
Input	Zod schemas parse body, path, and query data	Rejects malformed input before repository access
Auth	Bearer access token plus refresh-session flow	Keeps short-lived authorization separate from renewal
Success	Typed JSON resource or operation result; downloads expose filename	Predictable client handling
Client error	HTTP status plus stable code, message, and optional safe validation details	Enables useful UI without exposing internals
Unexpected error	Generic internal-error code with a correlated server log	Avoids database or stack disclosure
Pagination	Bounded page/limit or module-specific period	Controls response size and invalid extreme requests
Idempotency	Stable clientRequestId on supported replayable creation	Prevents duplicate logical creation

4.12 Security Design

Table 4.5 summarizes security controls and their purpose.

Table 4.5 Security controls and their purpose

Control	Protected risk	Evidence boundary
Password hashing and normalized identity	Credential disclosure and duplicate account variants	Strength still depends on deployment policy and user password

Control	Protected risk	Evidence boundary
Short access token plus opaque refresh session	Long-lived bearer exposure	Client and server must use HTTPS in production
Email verification and one-time challenges	Unverified ownership and replay	Production email deliverability is not yet verified
RBAC and ownership checks	Horizontal or vertical unauthorized access	Requires route-by-route maintenance and review
Trusted device and WebAuthn for privileged use	Admin credential takeover	Current browser test uses a virtual authenticator
Zod validation and bounded queries	Malformed data and resource abuse	Does not replace database constraints
Rate limits	Brute force and request flooding	Distributed behavior depends on configured Redis
Security headers and origin policy	Common browser attack surface	Final values must match the production domain
Atomic transactions and uniqueness	Partial or duplicate reward state	Verified by database lane when PostgreSQL test environment runs
Generic error responses and structured logs	Internal detail disclosure	Logs require secure retention and access policy

The realtime boundary now applies authorization as well as authentication. A requested guild room is joined only after a current GuildMember lookup succeeds; non-members and repository failures fail closed and receive a denial event. Focus presence is published only to the owner and accepted accountability peers. Production deployment must still replace the permissive local Socket.IO origin with the approved HTTPS origin and retain negative authorization tests.

4.13 Deployment and Operations Design

Figure 4.12 illustrates local deployment topology.

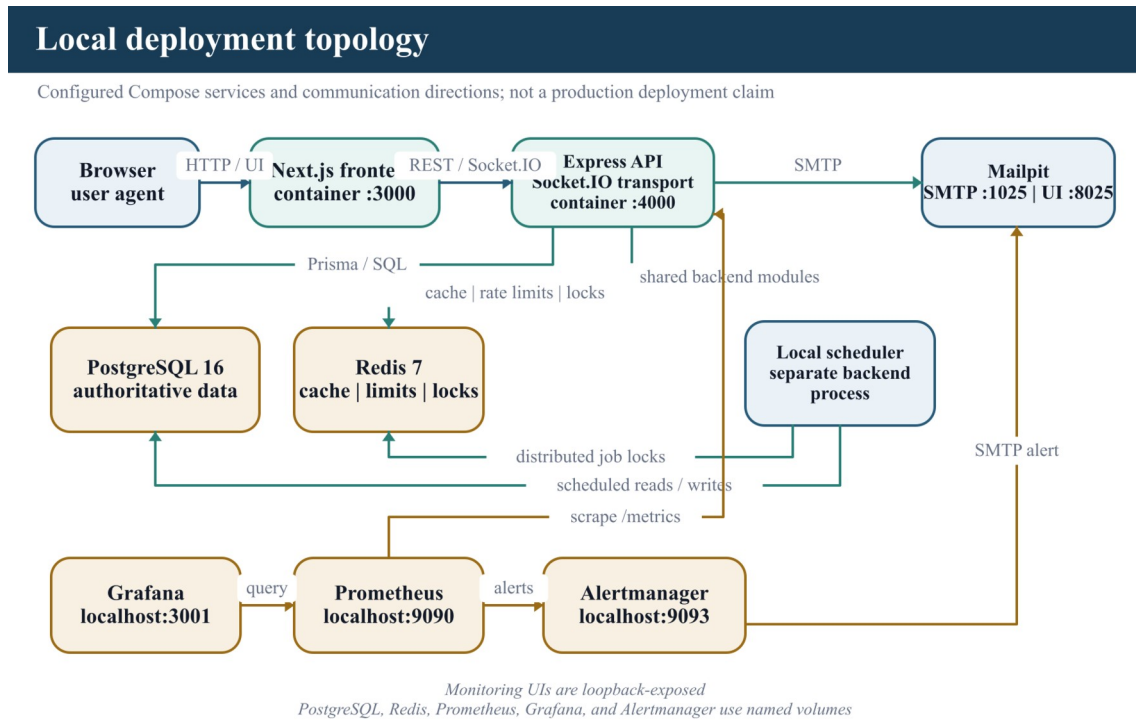


Figure 4.12 Local deployment topology

The supplied local topology separates frontend, API, scheduler, PostgreSQL, Redis, Mailpit, Prometheus, Alertmanager, and Grafana processes. The database is authoritative; Redis and monitoring services have narrower operational roles. Docker Compose supplies the declarative multi-service model [24]. Health and metrics routes help an operator observe the system, but the current database readiness label checks configuration rather than executing a database query. Production readiness should add live dependency probes, secret management, TLS termination, backups, restoration drills, external uptime checks, provider credentials, and an incident procedure.

The repository deployment workflow is manually triggered and validates/builds the applications. Its deployment hooks are disabled by default, and it does not itself prove that tests, migrations, or a provider-specific release occurred. I therefore describe deployment as locally orchestrated and production-deferred. Chapter VI records the verification commands, while the repository example environment files define the non-secret runtime contract.

4.14 Design Rationale and Trade-offs

- A modular monolith was preferred to microservices because the domain needs cross-module transactions and the current scale does not justify independent operational ownership.
- PostgreSQL is authoritative because reward, membership, session, and moderation relationships need constraints and transactions.
- Redis remains optional for supported accelerators and locks so local correctness does not depend on cached state.
- Realtime events request interface refreshes; they do not replace durable commands or records.
- The access token remains in memory to reduce persistent browser credential exposure, accepting that a refresh is needed after navigation or reload conditions.
- Offline scope is intentionally narrow because balance, completion, and social conflicts need current server authorization.
- Rule-based insights remain explainable and bounded because no validated training dataset or clinical assessment is available.
- Administrative friction is higher than user-login friction because the impact and threat model are different.

CHAPTER V

IMPLEMENTATION

5.1 Steps for Implementation

I implemented LevelUpX in controlled increments so that each user-facing capability rested on validated domain and persistence rules. The sequence below is also the reproducible local implementation procedure. It begins with locked dependencies and an isolated database, then progresses through verification, application startup, scheduled work, and operational checks.

1. Copy the backend and frontend example environment files and supply non-placeholder local secrets.
2. Install locked dependencies in each package with `npm ci`.
3. Start PostgreSQL and optional Redis, then apply Prisma migrations and seed only the intended environment.
4. Run backend and frontend type checks, zero-warning lint commands, unit tests, and production builds.
5. Run the guarded HTTP integration suite; provision a safely named isolated test database before database tests.
6. Start the backend and frontend on configured ports and run the Chromium browser suite with isolated fixtures and Mailpit when the complete environment is available.
7. Start the scheduler only once for each environment and verify recurring-quest uniqueness, job locks, and provider configuration.
8. Inspect health, protected metrics, logs, exports, delivery behavior, and administrator audit records before accepting real user traffic.
9. For production, apply TLS, restricted origins, managed secrets, backups, restoration checks, external monitoring, and a rollback plan.

5.2 Implementation Issues

Table 5.1 summarizes implementation issues and resolutions.

Table 5.1 Implementation issues and resolutions

Issue	Implemented response	Remaining limitation
Duplicate or concurrent quest completion	Guarded state transition, unique completion evidence, and one Prisma transaction	Requires the relational database lane to verify the deployed database and migration set
Completion-time consistency	The same-user transaction locks the profile row before calculating reward, UTC streak, and remaining daily coin allowance, then writes profile and ledgers together	UTC-day semantics are deliberate; a future user-timezone policy needs migration and boundary tests
Reward-scaling consistency	Difficulty is applied when stored quest rewards are drafted; completion applies streak and economy settings only	Economy changes still require audit and balance simulation
Coin inflation control	Administrator inflationRate reduces coin issuance through a bounded multiplier before caps	The parameter is a policy control, not a measured real-world inflation model
Skill-tree ordering	Schema prerequisites, service ordering, repository guards, and locked-state UI prevent dependent skills from advancing early	The built-in tree is intentionally small and should be expanded only with progression testing
Unauthorized guild subscription	Socket connection verifies the JWT and queries GuildMember before joining; lookup failures and non-members fail closed	Production Socket.IO origins must replace the current permissive local origin setting

Issue	Implemented response	Remaining limitation
Focus-presence privacy	Lifecycle events target the owner and accepted peers; blocking/removing a connection emits IDLE and the UI clears cached peer state	Presence is supplemental and may be temporarily stale after a delivery failure
Weekly team-quest validity	UTC series keys and a uniqueness guard create one current/future instance, skip missed historical weeks, reject progress outside the active window, and count malformed rows	Scheduler ownership and lock behavior still require deployment verification
Misleading workload guidance	Every BURNOUT_WARNING is normalized to fixed non-diagnostic workload/recovery copy, and provider scores are clamped to the range 0 to 1	Guidance scores are not calibrated probabilities or medical assessments
Rapid or repetitive completion patterns	Automatic rule-keyed reports flag at least ten hourly completions or five repeated titles in one day; a partial unique index prevents concurrent duplicate OPEN signals	Unexpected post-commit audit failures may reject the HTTP response even though the quest reward remains committed
Local infrastructure unavailable during final check	All available non-database lanes were rerun and unavailable lanes are reported separately	The 16 database tests and browser workflows require a provisioned environment

5.3 Algorithms

5.3.1 Quest Reward Calculation

Figure 5.1 illustrates reward calculation stages.

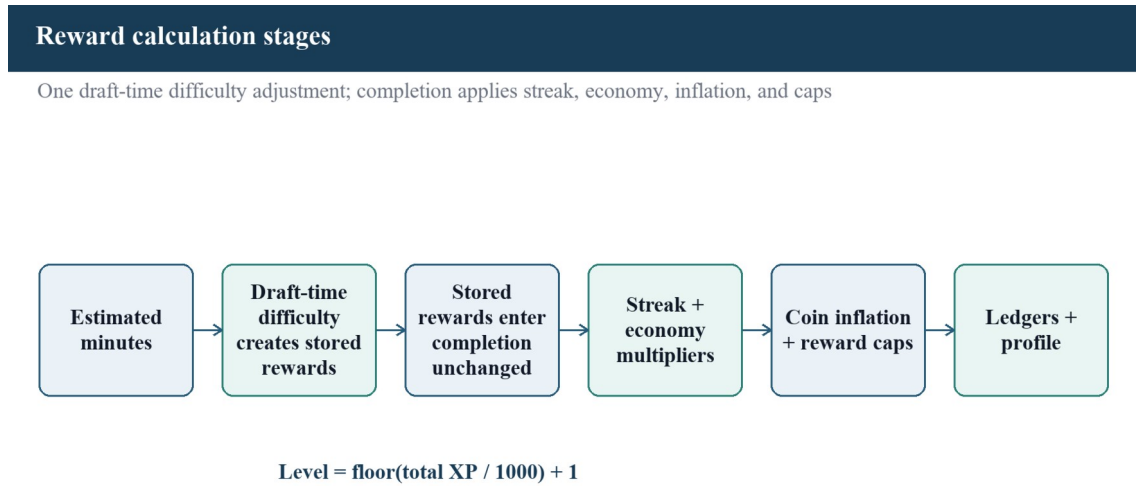


Figure 5.1 Reward calculation stages

Drafting a quest converts estimated minutes to base XP and coin values, then applies the selected difficulty once and stores the resulting rewards with the quest. At completion, the reward service uses those stored values. XP is multiplied by the stepped streak factor and administrator XP multiplier. Coins are multiplied by the administrator coin multiplier and a bounded issuance factor derived from inflationRate. Per-quest maxima are then applied, and coins are limited by the user's remaining daily allowance. Difficulty is not applied a second time. Table 5.2 summarizes quest difficulty multipliers.

Table 5.2 Quest difficulty multipliers

Difficulty	Multiplier	Interpretation
RECOVERY	0.75	Low-pressure recovery action
EASY	1.00	Normal base reward
MEDIUM	1.25	Moderate complexity
HARD	1.75	High effort
BOSS	2.50	Exceptional bounded challenge

Algorithm 5.1 Reward calculation pseudocode

```

difficulty = difficultyFactor(quest.difficulty)
storedXp = round(max(10, round(estimatedMinutes * 2)) * difficulty)
storedCoins = round(max(2, round(estimatedMinutes / 10)) * difficulty)
streakFactor = currentStreak >= 30 ? 2.0
                : currentStreak >= 14 ? 1.5
                : currentStreak >= 7 ? 1.25 : 1.0
xp = min(round(storedXp * streakFactor * xpMultiplier), maxQuestReward)
issuance = max(0, 1 - inflationRate / 100)
cappedCoins = min(round(storedCoins * coinMultiplier * issuance),
maxQuestReward)
coins = min(cappedCoins, max(0, dailyCoinLimit - coinsEarnedToday))

```

5.3.2 UTC Streak Maintenance

Quest completion calculates progression and streak state inside the same-user transaction. The repository locks the profile row before reading the latest completion and coins earned today, so concurrent requests cannot calculate from the same stale streak or daily allowance. A first completion sets the streak to one; another completion on the same UTC date preserves it; the next UTC date increments it; and a longer gap resets it to one. The longest streak, balances, completion, and reward ledgers are written before commit.

Algorithm 5.2 UTC streak projection pseudocode

```

elapsedDays = utcDay(completedAt) - utcDay(latestCompletionAt)
nextStreak = noPreviousCompletion ? 1
                : elapsedDays <= 0 ? max(1, currentStreak)
                : elapsedDays == 1 ? max(1, currentStreak) + 1
                : 1
longestStreak = max(longestStreak, nextStreak)

```

5.3.3 Level Derivation

The displayed level is derived as $\text{floor}(\text{total XP} / 1000) + 1$ and is updated in the completion transaction. XP ledger rows retain the reason, amount, source, and effective multiplier, allowing the current presentation state to be reconciled against its transaction evidence.

5.3.4 Skill Prerequisite Resolution

The built-in skill catalogue forms a directed prerequisite tree. Quest Planning is the root; Deep Focus requires Quest Planning level 1; Streak Discipline requires Quest Planning level 2; Guild Support requires Streak Discipline level 1; and Recovery Rhythm requires Deep Focus level 2. The service visits prerequisites before dependants, derives each eligible level from tracked activity, and the repository repeats the prerequisite check inside its transaction. Locked skills expose the required and current prerequisite levels to the profile and skills interfaces.

5.3.5 Idempotent Offline Replay

Algorithm 5.3 Offline replay pseudocode

```
queue(payload, clientId):  
    store {method, path, payload, clientId, createdAt} in IndexedDB  
    do not store the bearer token  
  
replay(action):  
    require a current in-memory session  
    send payload with the same clientId  
    if the server confirms success or the same logical creation:  
        delete the queued action  
    else if the error is transient:  
        keep the action for another retry  
    else:  
        expose the rejected action for resolution
```

5.3.6 Recurring Job Execution

Figure 5.2 illustrates background job and notification flow.

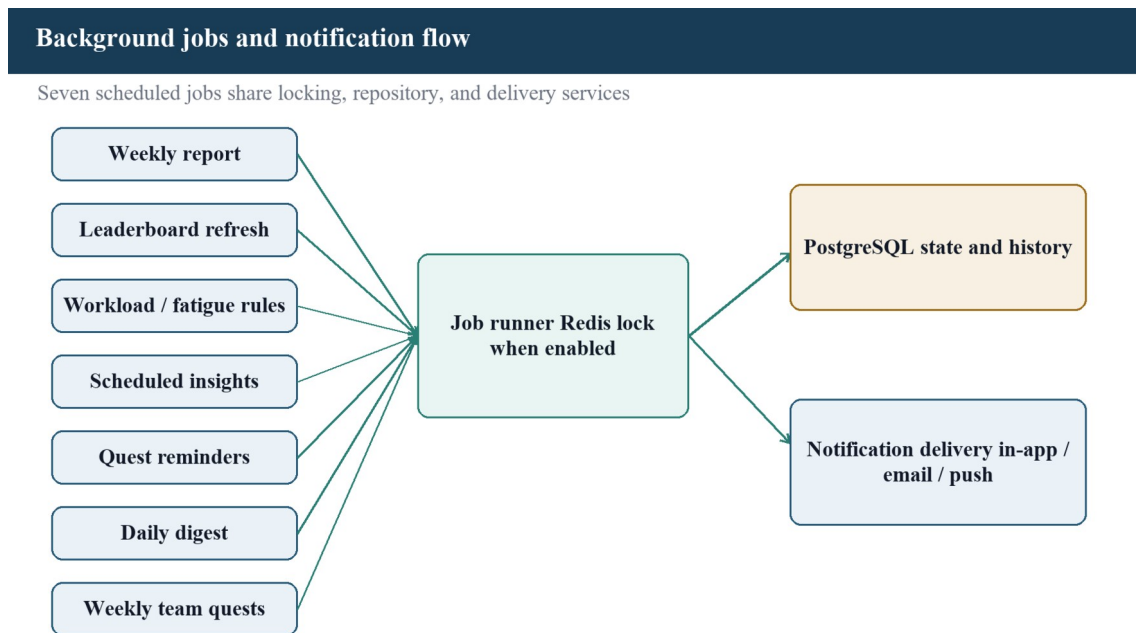


Figure 5.2 Background job and notification flow

The job runner attempts the configured lock for each stateful task, executes its domain service, records a structured result, and releases the lock. Weekly team-quest materialization groups recurring rows by series, selects the latest week, advances to the current or next eligible UTC week without backfilling missed historical weeks, and relies on compound series/week uniqueness for idempotency. Malformed recurrence rows are counted in the job result, and progress updates are rejected outside start/end dates. Table 5.3 summarizes background jobs.

Table 5.3 Background jobs

Job	Default hosted cadence	Purpose
leaderboard-refresh	Every 15 minutes	Refresh or materialize ranked views
burnout-prediction (compatibility job name)	Daily at 02:00 UTC	Create bounded, non-diagnostic workload and recovery guidance

Job	Default hosted cadence	Purpose
scheduled-insights	06:00 and 18:00 UTC	Generate eligible insight records under rollout/provider controls
quest-reminders	Every 5 minutes	Identify and deliver due reminder messages
daily-digest	Hourly check	Deliver timezone-aware daily summaries when due
weekly-report	Monday at 08:00 UTC	Prepare and deliver the previous-week summary
weekly-team-quests	Hourly at :05 UTC	Materialize one due UTC-week instance per recurring guild series

5.4 Implementation Strategy

I implemented LevelUpX incrementally, beginning with the database and identity foundation, then adding productivity workflows, progression, social modules, analytics, administrative controls, background jobs, offline capture, observability, and verification. At each stage I kept validation and service rules close to the affected domain, used migrations for relational changes, and added tests around high-risk behavior. This reduced the chance that a later interface feature would define business rules only in the browser.

The workspace contains separate frontend and backend packages. The frontend consumes the API rather than importing backend implementation files. The backend compiles as strict TypeScript and registers module routers from a central application factory. A separate scheduler entry point uses the registered job services. Local Compose configuration joins the processes with PostgreSQL, Redis, Mailpit, and the monitoring stack. The build is reproducible from package lockfiles, but production provider selection is intentionally not embedded in source. Table 5.4 summarizes implementation stages.

Table 5.4 Implementation stages

Stage	Implemented work	Verification emphasis
1. Foundation	Repository layout, TypeScript, environment parsing, Prisma schema, migrations	Configuration and schema checks
2. Identity	Registration, verification, login, recovery, sessions, roles	Validation, token/session, one-time challenge tests
3. Productivity	Projects, quests, completion transaction, focus sessions	Ownership, lifecycle, concurrency, rollback
4. Progression	Rewards, ledgers, classes, skills, achievements, cosmetics	Arithmetic, caps, uniqueness, balance consistency
5. Social	Guilds, team quests, accountability, leaderboards	Membership and payout rules
6. Reflection	Analytics, rule-based insights, notifications	Period calculations, preferences, bounded language
7. Administration	Control room, roles, economy, moderation, reports, security	RBAC, device binding, safe errors, audits
8. Delivery	PWA assets, offline queue, jobs, Compose, metrics, CI validation	Replay idempotency, builds, operational configuration

5.5 Frontend Implementation

The frontend uses the App Router to implement 23 page.tsx files covering public, authentication, onboarding, user, and administrative views. Shared layouts provide navigation and visual consistency. Interactive modules are client components that call typed API helpers, retain local form or timer state, and render a loading, success, empty,

or failure state. Zustand holds the authenticated in-memory session, while browser local storage is avoided for the bearer access token.

The user journey starts with registration and email verification, continues through onboarding, and enters a dashboard with quick quest capture and summarized activity. Dedicated pages expose quests, focus, profile, skills, achievements, rewards, guilds, leaderboard, analytics, insights, notifications, account/safety controls, a dedicated administrator login entry, and administration. The focus interface supplies a 25-minute Pomodoro-style work preset, a 60-minute deep-work preset, and separate 5- and 15-minute break presets. The current code does not automatically chain a Pomodoro cycle; each session is explicitly selected and controlled. Table 5.5 summarizes frontend responsibilities.

Table 5.5 Frontend responsibilities

Concern	Implementation	Important boundary
Routing	Next.js file-based pages and layouts	Authorization must also be enforced by the API
Session	In-memory Zustand access state plus server refresh flow	No bearer token in localStorage or offline queue
Forms	Controlled inputs and API validation feedback	Backend Zod schemas remain authoritative
Timer	Local elapsed/pause state synchronized through focus API actions	Elapsed time is not proof of task quality
Charts/ motion	Recharts user analytics, Chart.js administrator analytics, and Framer Motion login transitions	No formal chart-comprehension or motion-usability study
Realtime	Socket refresh events plus accepted-peer focus presence	Missed events are repaired by REST reads
Offline	IndexedDB queue for quick quest capture	Not a general offline mutation layer
PWA	Manifest, icon, offline fallback, service worker	Navigation remains network-first

5.6 Backend Implementation

The backend creates an Express application through a factory so tests can instantiate the HTTP boundary without opening a production listener. Middleware establishes request context, validates origins and payload size, adds security headers, logs structured request results, authenticates bearer tokens, and converts failures into a stable public error contract. Module routers then delegate to controllers, services, and repositories.

Zod schemas normalize and bound external input. Services throw typed application errors for expected states such as authentication required, forbidden action, missing resource, or conflict. Repositories translate domain requests to Prisma. This division allowed the 270 tests in 53 files in the default backend Vitest suite to mock data or delivery collaborators. Sixteen isolated database tests are also present, but they were skipped in the final check because `TEST_DATABASE_URL` was not configured. Table 5.6 summarizes backend layer responsibilities.

Table 5.6 Backend layer responsibilities

Layer	Responsibility	Example
Route	Mount verb/path and middleware	GET /api/rewards/summary requires authentication
Validation	Parse body, path, and query	Normalize register email and reject short password
Controller	Translate request and response	Return JSON or export with Content-Disposition
Service	Apply business rule and coordinate dependencies	Calculate reward or block self-role change
Repository	Read/write authoritative data	Prisma transaction for quest completion
Provider	Send or integrate with an external capability	SMTP, push, insight provider, realtime publisher
Middleware	Cross-cutting boundary enforcement	Role, device session, rate limit, metrics token
Job	Initiate scheduled domain work	Reminders, digests, reports, leaderboard refresh

5.7 Database and Migration Implementation

5.7.1 Ownership and Relational Modeling

Prisma expresses enumerations, columns, defaults, uniqueness, indexes, and relations for the 39 models. Migration files provide an ordered relational history. Repository methods select only the data required by a use case and use transactions when one logical operation changes several records. The test database helper refuses to reset a database unless the environment is test and the database name contains a test marker, reducing the risk of destructive execution against a development or production database.

User is the ownership root for personal records. UserProfile is one-to-one presentation state, while XP and coin transaction rows retain the reason behind calculated totals. QuestCompletion is separated from Quest and limited to one record per quest. Join models represent many-to-many relationships such as guild membership, earned achievements, acquired skills, owned cosmetics, team contributions, and push subscriptions. Nullable quest/project links allow focus sessions or quests to exist independently where the workflow permits it; service authorization still prevents one user from attaching another user's data.

5.7.2 Transactions and Concurrency

Quest completion is the clearest transaction boundary. The repository verifies the current state, creates one QuestCompletion, updates the Quest, writes XpTransaction and CoinTransaction rows, and updates UserProfile balances, level, and UTC streak state. A failure in any step rolls the entire operation back. The database-test source includes concurrent completion and rollback cases, but that lane was skipped in the final check because no isolated test database was configured.

The current implementation locks the same user's profile row before deriving the completion streak and remaining daily coin allowance. This prevents two different quest completions for the same user from independently consuming the same allowance or calculating from the same previous streak. Other guarded mutations use conditional updates, unique constraints, or transaction isolation: refresh rotation replaces one matching unexpired session row, one-time challenges are consumed once, weekly team quests use a recurrence-series/week key, and automatic abuse signals use a stable rule key with an OPEN-report partial unique index. These controls protect database state; they do not remove the need to rerun the relational tests on PostgreSQL before release.

5.7.3 Migration and Data-Safety Rules

Each schema change is represented by an ordered migration rather than an application-startup schema rewrite. Development may create a migration, while deployment should apply only reviewed existing migrations. A production pipeline must back up data, test the migration on staging, retain a rollback or forward-fix plan, and verify the application after the change. The supplied workflow does not currently perform those production steps, so migration files are implementation evidence rather than proof of a successful hosted upgrade. Table 5.7 summarizes critical relational invariants and enforcement.

Table 5.7 Critical relational invariants and enforcement

Invariant	Enforcement and protected operation	Current evidence boundary
One normalized account email	Unique User.email after service normalization protects registration and account lookup.	Current unit and early HTTP validation evidence.
One profile per user	Unique UserProfile.userId plus owned repository access protects level, balance, streak, and class state.	Current units; relational suite skipped.
One payout record per quest	Unique QuestCompletion.questId inside the guarded transaction protects quest state, XP/coin ledgers, and profile update.	Current unit/source evidence; concurrent DB cases skipped.
One logical offline capture	Stable Quest.clientRequestId uniqueness protects supported reconnect/retry quest creation.	Current frontend/backend units; browser not rerun.
Serialized same-user reward state	A profile-row lock before streak, allowance, and payout calculation protects concurrent completion of different quests.	Source and unit evidence; PostgreSQL lane skipped.
Prerequisite-gated skill progress	A Skill self-relation and repository/service prerequisite check protect unlock and advancement.	Current prerequisite unit evidence.

Invariant	Enforcement and protected operation	Current evidence boundary
One weekly recurrence instance	Compound series/week uniqueness with conflict handling protects scheduled materialization and retries.	Current job units; relational recurrence case skipped.
One OPEN automatic signal per rule key	A stable dedupe key and PostgreSQL partial unique index protect rapid/repetitive completion audit races.	Current service units; DB concurrency case skipped.

5.8 Authentication and Security Implementation

Registration normalizes the email address and stages a hashed verification challenge. Successful verification activates the account and establishes a normal user session. Login verifies the password and, where configured, an additional email challenge. Refresh uses a stored opaque-session record rather than trusting a permanently reusable browser value. Logout or session revocation invalidates that record. Password recovery returns a generic response so an unauthenticated caller cannot enumerate accounts.

The frontend exposes `/admin/login` as a distinct privileged entry while preserving the same staged security model. A separate administrative CLI pre-provisions a trusted device and its key. Privileged login proves the pre-provisioned device key and the required email challenges to obtain a base verified admin session; a subsequent WebAuthn step-up produces the passkey-verified session accepted by admin routes. Role middleware controls route access, while the admin-security middleware checks token claims and the current active device record. Service rules block unsafe operations such as a super administrator changing the role of the same account. Metrics can require a distinct bearer secret. The centralized error handler exposes stable codes for expected conditions and hides unexpected implementation text.

5.9 Analytics and Insight Implementation

Analytics repositories aggregate the authenticated user's stored quests, focus sessions, rewards, and activity into bounded time periods. The frontend presents summaries, trends, a heatmap, and related action links. These views are descriptive: they show what the application recorded and do not establish causation or work quality.

The default workload and fatigue guidance is rule based. It evaluates recent stored patterns and creates suggestions such as scheduling recovery work or reducing an overloaded plan. An optional external insight provider contract exists with rollout and fallback controls, but no trained proprietary model is part of the submitted project. Every BURNOUT_WARNING is replaced with fixed non-diagnostic workload/recovery copy, and provider scores are validated and clamped to the range 0 to 1. Other optional external insight types retain provider title/message text after structural and length validation, while external analytics recommendations are trimmed and limited but not fully semantic-filtered. Prompt-run history supports operational review when provider-assisted generation is enabled. Hosted use therefore requires provider governance and output review. A displayed score is a bounded guidance value, not a statistically calibrated probability, medical risk, or measured model accuracy.

5.10 Realtime and Notification Implementation

The realtime publisher uses stable user and guild room names. Important state changes are persisted first; a socket event then tells interested interfaces that a view may be refreshed. Notification records and per-user preferences separate durable inbox state from delivery channels. Email, web push, and local Mailpit behavior are controlled by environment configuration. Production sender reputation, push keys, and delivery monitoring were not available for this report.

Guild REST services and the socket boundary both enforce membership. The realtime server verifies the access token, queries current GuildMember state before joining, and fails closed for a non-member or lookup error. Focus lifecycle changes publish FOCUSING, PAUSED, or IDLE presence only to the owner and accepted accountability peers. Blocking or removing a connection emits an IDLE state and the frontend clears the related cached presence so a former peer is not left visible. These realtime signals remain supplemental to durable REST state.

5.11 Navigation and Interface Implementation

Figure 5.3 illustrates implemented navigation structure.

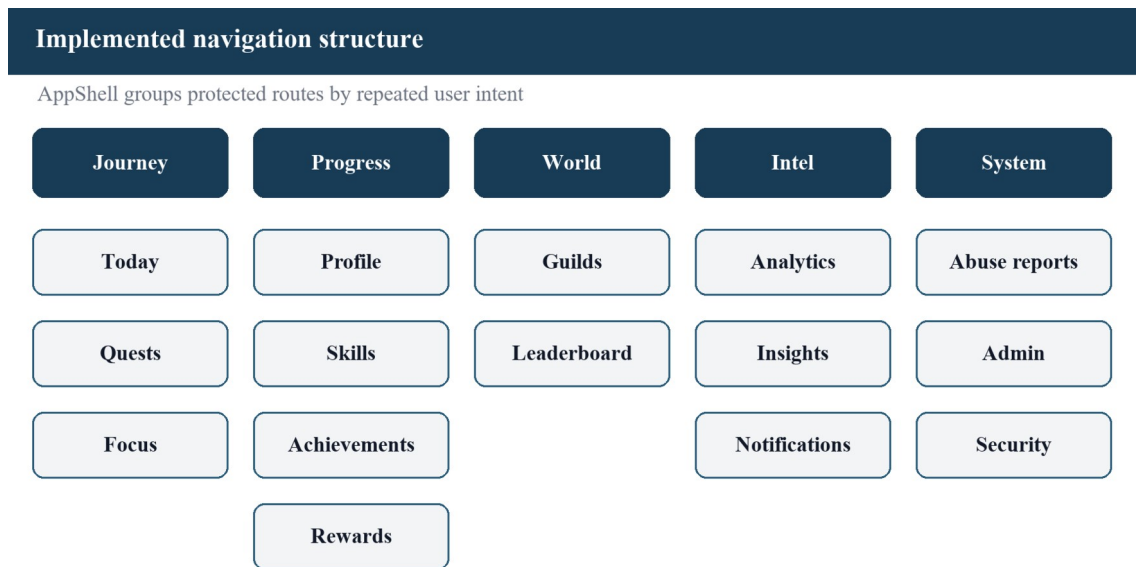


Figure 5.3 Implemented navigation structure

The implemented navigation groups daily productivity, progression, social work, reflection, and system actions. This reduces a long feature list into recognizable destinations. The profile is the bridge between identity and progression; rewards combine ledgers, cosmetics, and personal redemption; the admin route separates ordinary user navigation from privileged operations. Empty states and calls to action guide an initial user toward the next supported workflow.

CHAPTER VI

TESTING

Testing Objectives and Strategy

I used layered testing because no single command can establish the behavior of the whole system. Vitest supplies the primary unit/integration runner [25], while Playwright supplies browser automation [26]. Unit tests isolate validation, services, providers, middleware, utilities, state stores, and components. A dedicated HTTP integration suite creates the Express application and checks routing, middleware, CORS, validation, and public error responses. Database tests use an isolated PostgreSQL database for transactions and concurrency. Playwright drives critical browser journeys against the frontend, backend, database, and local email capture service. Type checking, linting, and production builds provide separate static and packaging evidence.

The evidence in this chapter distinguishes a current pass from a skipped or unavailable lane. Current means I ran the command successfully against the final adherence source on 12 August 2026. Skipped means the tests were collected but their required isolated database was not configured. Not rerun means the browser environment was unavailable after the adherence changes. This distinction prevents source existence or an older project record from being misreported as a current pass. Figure 6.1 illustrates testing layers used in the project.

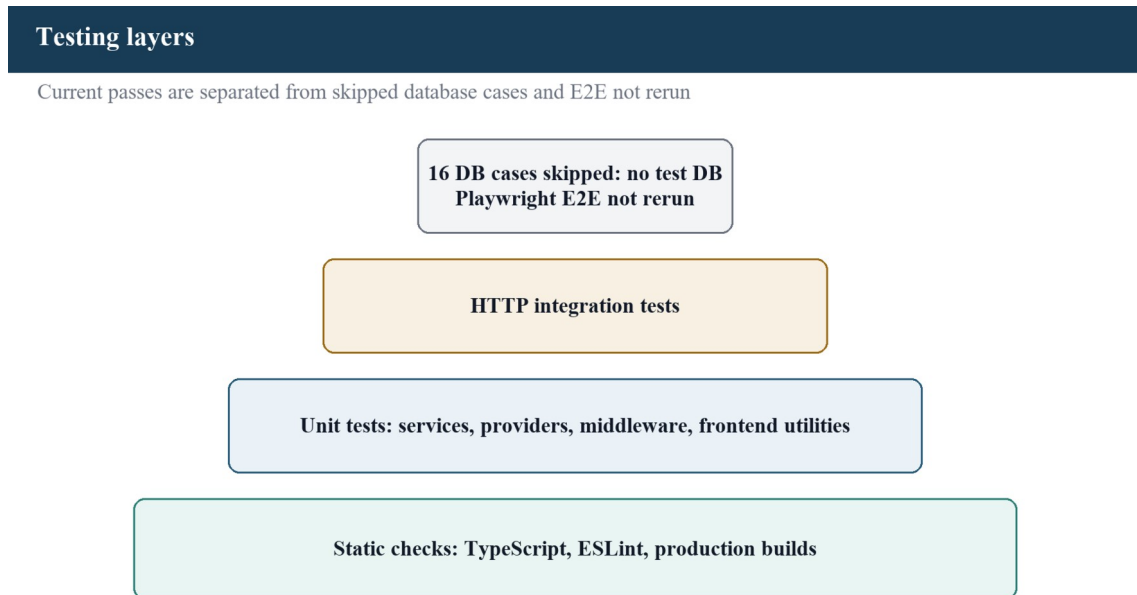


Figure 6.1 Testing layers used in the project

6.1 Test Environment

Table 6.1 summarizes current verification environment.

Table 6.1 Current verification environment

Item	Observed value	Use
Host	Windows NT 10.0.26200	Final non-database verification host
Node.js	22.16.0	Backend, frontend, test, and build runtime
npm	11.7.0	Package-script runner
Backend test runner	Vitest 4.1.6	Default and dedicated backend suites

Item	Observed value	Use
HTTP test client	Supertest 7.2.2	In-process Express request assertions
Frontend test runner	Vitest 4.1.10 + jsdom 29.1.1	Store, queue, and component tests
Browser framework	Playwright 1.62.1, Chromium project	Critical workflow source; not rerun after adherence changes
ORM	Prisma Client 5.22.0	Repository access and database suite
Docker CLI	29.6.1; engine unavailable	Reason test database was absent and Playwright was not rerun
Repository history	No .git metadata in workspace	No commit-authenticated history or current remote CI status

6.2 Unit Testing of Modules

The unit tests concentrate on decisions that can regress without a visible compilation error: identity normalization, one-time challenge lifecycle, reward arithmetic, achievement uniqueness, membership and payout rules, UTC streak transitions, skill prerequisites, guild room authorization, accepted-peer focus presence, weekly recurrence, abuse signals, PDF/CSV exports, rate limiting, safe error serialization, offline credential handling, and component behavior. Repository and provider doubles allow exact edge conditions to be injected. These tests are fast and deterministic, but mocks cannot prove that a real PostgreSQL constraint or browser interaction behaves identically. Table 6.2 summarizes representative unit tests.

Table 6.2 Representative unit tests

ID	Area and input	Expected result	Current evidence
UT-01	Register name/email contain outer spaces and uppercase email	Trim name; normalize email; preserve password	CURRENT BACKEND PASS
UT-02	New registration then correct eight-digit verification code	No session before verification; verified USER session after	CURRENT BACKEND PASS

ID	Area and input	Expected result	Current evidence
UT-03	Stored 100 XP/40 coins; streak 7; settings 1.2/1.5	150 XP, 60 coins, multiplier 1.5; no repeated difficulty	CURRENT BACKEND PASS
UT-04	Inflation 20%; coin multiplier 1.5; stored coins 50	Coin issuance is reduced to 60	CURRENT BACKEND PASS
UT-05	Same-day, next-day, and multi-day-gap completions	Streak preserves, increments, or resets under UTC-day rules	CURRENT BACKEND PASS
UT-06	Dependent skill below prerequisite	Skill stays locked and reports required/current prerequisite levels	CURRENT BACKEND PASS
UT-07	Guild room request by member, non-member, and failed lookup	Only membership success returns guild room; others fail closed	CURRENT BACKEND PASS
UT-08	Focus lifecycle, then block/remove an accepted peer	Presence targets self/accepted peers; block/remove emits IDLE and clears client cache	CURRENT BACKEND + UI PASS
UT-09	Weekly duplicate/missed weeks, malformed row, and progress outside dates	One eligible instance; no historical backfill; malformed count; invalid progress rejected	CURRENT BACKEND PASS
UT-10	Ten completions in one hour or five repeated titles	Deduplicated automatic abuse review reports are created	CURRENT BACKEND PASS
UT-11	Analytics and marketplace export as CSV and PDF	Escaped CSV and %PDF-1.4 output with stored metadata	CURRENT BACKEND PASS

ID	Area and input	Expected result	Current evidence
UT-12	Provider insight score outside 0 to 1	Score is clamped and workload copy remains non-diagnostic	CURRENT BACKEND PASS
UT-13	Admin token refers to a revoked/missing trusted device	401 stale-device-session error	CURRENT BACKEND PASS
UT-14	Browser UI sets then clears an authenticated session	Token stays in memory; logout clears all session state	CURRENT FRONTEND PASS
UT-15	Queue offline POST /quests then replay succeeds	Credential is not persisted; action is removed after success	CURRENT FRONTEND PASS
UT-16	Use dedicated administrator login entry	Admin entry preserves the staged privileged authentication flow	CURRENT FRONTEND PASS

6.3 Integration Testing of Modules

The dedicated integration suite instantiates the application and sends real HTTP requests through the configured middleware chain. Database access is avoided in the selected cases so routing and boundary behavior can run without PostgreSQL. The five current passing tests cover health, not-found behavior, early validation, authentication rejection, and a download-related CORS header. Table 6.3 summarizes integration and database test evidence.

Table 6.3 Integration and database test evidence

ID	Lane and scenario	Expected result	Execution status
IT-01	HTTP GET /api/health	200; service, status, API, and DB-config fields	CURRENT PASS

ID	Lane and scenario	Expected result	Execution status
IT-02	HTTP GET /api/missing-route	404 ROUTE_NOT_FOUND with method and path	CURRENT PASS
IT-03	Invalid register name, email, and password	400 validation error before database access	CURRENT PASS
IT-04	Protected reward summary without token	401 AUTH_REQUIRED	CURRENT PASS
IT-05	CORS preflight for account export	204; Content- Disposition is exposed	CURRENT PASS
DB-01-16	Four-file isolated Prisma transaction/concurrency suite	Completion, serialized streak/daily allowance, rollback, challenge, device, reset, recurrence, abuse-signal deduplication, and relational invariants	4 FILES / 16 COLLECTED; SKIPPED - NO TEST DB

6.4 System Testing

System verification combines the independently checked packages. Backend and frontend type checks prove that the final adherence source satisfies the configured static contracts. Zero-warning lint commands check the selected coding and framework rules. Production builds prove that the TypeScript backend emits runnable JavaScript and Next.js compiles the application graph into 25 statically generated pages. These checks do not establish live database, browser, provider, or capacity behavior, but they detect integration faults that isolated module tests may not import.

Docker Compose configuration was parsed successfully, but the final host did not provide the isolated PostgreSQL test database or complete browser topology. Therefore, current system evidence comprises the static checks, production builds, default/unit suites, and HTTP boundary suite; database and browser behavior remain explicitly separate in the result table.

6.5 Functional Testing

Functional cases connect requirements to end-user outcomes. The browser suite is valuable because it crosses page routing, JavaScript state, HTTP, PostgreSQL, SMTP capture, WebAuthn simulation, download behavior, and responsive rendering. I did not rerun these workflows after the adherence changes because Docker Desktop and the isolated multi-service environment were unavailable. The present source and older browser artifacts are therefore not reported as a current browser pass. Table 6.4 summarizes functional test cases.

Table 6.4 Functional test cases

ID	Workflow / test data	Expected result	Status
FT-01	Register, verify email, select mode/class, and create a first quest	Verified account reaches the dashboard and shows its first quest	SOURCE/ UNIT; E2E NOT RERUN
FT-02	Ordinary and administrator login entry routes	User entry continues normally; /admin/login initiates the staged privileged flow	SOURCE/ UNIT; E2E NOT RERUN
FT-03	Complete quests on the same UTC day, next day, and after a gap	Current/longest streak preserve, increment, or reset atomically	CURRENT UNIT; DB SKIPPED
FT-04	Subscribe to a guild room as a member and non-member	Member joins; non-member or lookup failure receives no room access	CURRENT UNIT; E2E NOT RERUN
FT-05	Start focus, then block or remove the accepted accountability peer	Presence reaches self/accepted peer; IDLE and cache clearing remove former-peer state	CURRENT UNIT; E2E NOT RERUN

ID	Workflow / test data	Expected result	Status
FT-06	Run weekly materialization after missed weeks and update progress outside its date window	Only an eligible instance is created; historical weeks are skipped; invalid progress is rejected	CURRENT UNIT; DB SKIPPED
FT-07	Export administrator analytics and marketplace history	CSV and valid PDF downloads are produced and report metadata is recorded	CURRENT UNIT; E2E NOT RERUN
FT-08	Create quest with required fields and start it	Quest moves from PENDING to IN_PROGRESS	Covered by source/unit; browser case needed
FT-09	Complete a started quest	Profile and ledger display the authoritative capped, inflation-adjusted reward	Covered by current unit; DB/browser rerun needed
FT-10	Pause and resume a focus session, add a note, stop as complete	History and statistics reflect net elapsed state	Covered by unit; browser case needed
FT-11	Join a guild and contribute to an authorized team quest	Guild progress and payout refresh from server state	Covered by unit; browser case needed
FT-12	Complete quests rapidly or repeat one title, including concurrent audit attempts	One OPEN rule-keyed review signal is retained; a duplicate does not roll back the committed quest reward	CURRENT UNIT; DB/browser rerun needed

6.6 Results

Table 6.5 summarizes verification command results on 12 August 2026.

Table 6.5 Verification command results on 12 August 2026

Area	Command	Result	Status
Backend default tests	npm test	53 files; 270 tests passed	CURRENT PASS
Backend HTTP integration	npm run test:integration	1 file; 5 tests passed	CURRENT PASS
Frontend tests	npm test	6 files; 10 tests passed	CURRENT PASS
Backend type check	npm run typecheck	TypeScript completed without error	CURRENT PASS
Frontend type check	npm run typecheck	TypeScript completed without error	CURRENT PASS
Backend lint	npm run lint	ESLint completed with zero warnings	CURRENT PASS
Frontend lint	npm run lint	ESLint completed with zero warnings	CURRENT PASS
Backend build	npm run build	TypeScript production output created	CURRENT PASS
Frontend build	npm run build	Next.js production build; 25 static pages generated	CURRENT PASS
Database tests	npm run test:db	4 files; 16 tests collected and skipped because TEST_DATABASE_URL was not configured	SKIPPED / NO TEST DB
Browser tests	npm run test:e2e	Not rerun after the adherence changes because the full multi-service environment was unavailable	NOT RERUN

The backend default configuration excludes integration and database suffixes, so its 270 passing tests do not replace the five HTTP tests or the 16 database cases. The frontend unit suite runs in jsdom. The source contains 23 page.tsx files, while the successful production build generated 25 static pages. The browser configuration remains available, but it produced no current post-adherence execution evidence.

6.7 Security-Focused Testing

OWASP ASVS was consulted as a security-test planning reference [27]. I do not claim ASVS conformance because the project has not completed control-by-control verification.

- Validation tests cover normalized identity, minimum credentials, bounded pagination, identifiers, and malformed payload rejection.
- Authentication tests cover unverified registration, one-time challenges, refresh/session lifecycle, password recovery, and generic unauthenticated responses.
- Authorization tests cover missing tokens, role rejection, ownership, stale admin tokens, active trusted devices, and protected metrics.
- Integrity tests cover one-per-quest payout, rollback, concurrent completion, concurrent challenge consumption, and device-revocation invariants.
- Privacy tests verify that the offline queue does not persist the access token and that unexpected errors do not disclose database details.
- Guild-room authorization tests cover a current member, a non-member, and a repository failure; both negative paths fail closed.
- Presence tests verify that focus activity is sent only to the owner and accepted accountability peers.
- The privileged browser workflow still requires rerun with a physical passkey on the deployment domain.

6.8 Test Result Analysis

The current evidence is strongest for deterministic business rules, middleware boundaries, type safety, lint configuration, and compilation. Current unit coverage now includes the adherence additions: UTC streaks, prerequisite skills, inflation-adjusted rewards, fail-

closed guild subscription, accepted-peer presence, weekly team-quest recurrence, bounded insight output, administrator PDF/CSV exports, and automated abuse signals. Counts and implementation assertions were checked against the submitted source workspace [28].

The evidence is weaker for real production conditions. There is no current database or browser rerun, no Firefox or Safari project, no physical WebAuthn key, no hosted SMTP or push delivery, no formal load profile, no penetration assessment, and no participant usability or accessibility study. Therefore, a current local pass is not equivalent to production readiness, and source existence is not equivalent to a user-validated outcome.

6.9 Testing Limitations

Table 6.6 summarizes testing limitations.

Table 6.6 Testing limitations

Limitation	Effect on interpretation	Required follow-up
No isolated test database	All 16 database tests in 4 files were collected but skipped	Configure TEST_DATABASE_URL and rerun sequentially before release
Browser suite not rerun	No current post-adherence browser result exists	Run the complete Playwright topology and retain its report
No .git metadata	Cannot authenticate commits, diff, branch, or current remote CI state	Verify in the canonical repository and retain CI artifacts
Mocks in unit tests	Provider and repository integration may differ	Maintain relational, SMTP, push, and browser lanes
Chromium-only browser project	Firefox, Safari/WebKit, and device differences are unknown	Add cross-browser and physical-device matrix
Virtual passkey	Hardware, OS, and deployed-origin behavior is not proved	Test registered physical authenticators over HTTPS

Limitation	Effect on interpretation	Required follow-up
Mailpit	Local SMTP capture does not prove internet deliverability	Verify domain, SPF, DKIM, DMARC, bounce, and retry behavior
No load profile	Throughput, latency, pool sizing, cache value, and job overlap are unknown	Define traffic model and run load/soak tests
No formal accessibility evaluation	Semantic and visual intentions are not a conformance claim	Run automated and keyboard/screen-reader evaluation
No participant study	No productivity, motivation, comprehension, or satisfaction improvement can be claimed	Conduct consented evaluation with defined measures
Historical operational log	Earlier container/monitoring pass may not match current state	Rerun Compose, migrations, rules, metrics, dashboards, and alert delivery

6.10 Acceptance Summary

The available evidence meets the local academic acceptance threshold for implemented source, current non-database automated checks and compilation. It does not meet a production-operations acceptance threshold. Before public use, I would require a current clean run of the database and browser lanes, restricted production socket origins, hosted dependency checks, recovery evidence, broader browser and accessibility testing, and documented approval of secrets and operational policy.

CHAPTER VII

RESULTS AND ANALYSIS

7.1 Result Overview

The project result is a substantial working local implementation of the proposed gamified productivity platform. The repository contains a responsive Next.js application, an Express API, a 39-model Prisma schema, PostgreSQL migrations, a scheduler, real-time integration, offline quick capture, local container orchestration, monitoring configuration, and layered automated tests. The user interface contains 23 page.tsx source files, while the successful production build reported 25 statically generated pages. The backend exposes 130 HTTP handlers across identity, productivity, progression, social, analytics, insight, notification, safety, administration, health, and metrics route families.

These counts describe implementation breadth, not user benefit. A page count does not prove usability; an endpoint count does not prove security; and a passing test count does not prove production reliability. I therefore analyze the result against each stated objective and attach the relevant limitations rather than presenting aggregate counts as quality scores.

7.2 Results by Functional Module

Table 7.1 summarizes implemented results by module.

Table 7.1 Implemented results by module

Module	Result available in the project	Evidence boundary
Identity	Verified registration, login, optional user two-step, recovery, refresh, logout, and session controls	Hosted email delivery not verified
Privileged identity	Pre-provisioned trusted-device proof, three codes, passkey enrolment/step-up, device revocation	Historical browser test uses virtual authenticator

Module	Result available in the project	Evidence boundary
Onboarding	Mode, class, preferences, and first-quest journey	No completion-rate or usability study
Projects	Create, list, edit, and archive owned projects	No restore or hard-delete operation
Quests	Planning fields, filtering, edit, start, complete, fail, archive, recurrence, reminders, subtasks	Some API edit capability is not exposed on the current quest page
Focus	Work/break presets, custom target, pause/resume, note, stop result, history, statistics	No automatic Pomodoro cycle or attention measurement
Progression	Same-user locked calculation of UTC streak, reward, and daily allowance; single difficulty-adjusted rewards; inflation-adjusted coins; ledgers, levels, achievements, and prerequisite skills	Timezone policy is UTC and relational verification must be rerun
Marketplace	Browse, list, buy, cancel, purchase, and equip supported cosmetics	No payment or cash value; no unequip endpoint
Guilds	Create/join/leave, invite code, recurring weekly team quests, contributions, scoped leaderboard, and membership-gated realtime rooms	Broader browser and load behavior remain unevaluated
Accountability	Request, respond, block, list, remove, and share focus presence only with accepted peers; block/remove emits IDLE and clears cached presence	No participant evaluation of social effects

Module	Result available in the project	Evidence boundary
Analytics	Weekly summary, monthly heatmap, focus consistency, recommendations	Descriptive records; no causal productivity claim
Insights	Rule-based generation, study plan, schedule, learning, feedback, bounded guidance scores, non-diagnostic copy, and prompt-run records	Guidance scores are heuristics, not calibrated probabilities
Notifications	Inbox, unread state, preferences, push subscriptions, email/push provider hooks	Production delivery accounts are deferred
Safety/account	Manual reports, automated rapid/repetitive completion signals, personal data export, and permanent account deletion	Threshold governance, retention, and operator procedure need production approval
Administration	Dedicated admin login, dashboard, users, roles, economy, reports, moderation, marketplace, analytics, audit, system health, and CSV/PDF exports	Deploy-time role/policy decisions remain external
Operations	Scheduler, structured logs, health, protected metrics, and monitoring-stack configuration	Current live Compose status was not verified
PWA/offline	Manifest, offline fallback/static caching, IndexedDB quick capture, idempotent replay	All other mutations require the network/API

7.3 Objective Achievement Analysis

Table 7.2 summarizes objective achievement analysis.

Table 7.2 Objective achievement analysis

Objective	Result	Assessment and qualification
Verified identity and RBAC	Implemented and currently unit/HTTP verified	ACHIEVED LOCALLY; hosted email/passkey evidence remains
Projects and detailed quests	Implemented across API and main UI	ACHIEVED WITH UI/restore boundaries
Timed focus management	Implemented with explicit work/break sessions	ACHIEVED; no automatic cycle or outcome study
Consistent XP/coin progression	Atomic UTC streak maintenance, single difficulty application, inflation-adjusted coins, transactions, ledgers, levels, achievements, and skill prerequisites are present	ACHIEVED LOCALLY; database lane must be rerun
Analytics and bounded insights	Summaries, non-diagnostic rule-based guidance, and clamped guidance scores are present	ACHIEVED TECHNICALLY; scores are explicitly heuristic
Guild/social/accountability	Membership-gated rooms, weekly recurring team quests, ranking, accepted-peer focus presence, and connection lifecycle are present	ACHIEVED LOCALLY; broader browser/load evidence is pending

Objective	Result	Assessment and qualification
Administrative controls	Control-room route and staged security present	ACHIEVED LOCALLY; production policy/provider review pending
Limited offline PWA	Quick capture stores and replays without credentials	ACHIEVED FOR STATED NARROW SCOPE
Validation and security controls	Schemas, middleware, sessions, transactions, safe errors, rate limits	SUBSTANTIALLY ACHIEVED; not a penetration-test claim
Layered verification	270 backend tests in 53 files, 5 HTTP tests in 1 file, 10 frontend tests in 6 files, both typechecks/lints/builds, and a 25-static-page Next build passed	PARTIAL CURRENT EVIDENCE: 16 DB tests in 4 files skipped without a test database and E2E was not rerun

7.4 Quantitative Implementation Evidence

The implementation inventory provides reproducible structural evidence. I counted 39 Prisma models in the submitted schema and 130 Express route handlers in the registered module route files. The frontend has 23 page.tsx source files. The current backend default suite passed 270 tests in 53 files, the dedicated HTTP suite passed 5 tests in 1 file, and the frontend passed 10 tests in 6 files. Backend and frontend type checking, zero-warning linting, and production builds passed, and the Next.js build generated 25 static pages. The current database runner collected 16 tests in 4 files but skipped them because no isolated test database was configured; Playwright E2E was not rerun after the adherence changes. The dated 5 August record is retained only as historical context [29].

These values are useful for traceability because an examiner can locate the corresponding files and commands. They should not be combined into a percentage of completeness. The repository has no .git metadata in the supplied workspace, so I cannot bind the inventory to a commit identifier or verify the status of a remote continuous-integration run.

7.5 Result Interpretation

7.5.1 Productivity Result

LevelUpX successfully connects a planned quest, an optional focus record, a transaction-backed reward, and a later analytical view. This is the central engineering result. It reduces fragmentation at the data and workflow level. It does not establish that a user completes higher-quality work, retains a habit, or experiences greater intrinsic motivation. Those outcomes require a participant evaluation with measures that distinguish recorded activity from meaningful performance.

7.5.2 Integrity and Security Result

The strongest integrity result is the guarded quest-completion transaction and its database test design. The strongest identity result is the separation of ordinary authentication from privileged trusted-device, code, and passkey stages. Safe error behavior, role middleware, session re-checks, and test-database guards support these mechanisms. Realtime guild subscriptions now fail closed through a current GuildMember lookup, and focus presence is limited to the owner and accepted accountability peers. These controls reinforce why every transport boundary needs its own authorization decision; production origin restrictions and broader socket integration evidence are still required.

7.5.3 Offline and Realtime Result

The offline result is deliberately small and complete enough to reason about: one supported creation command, one local structured queue, no persisted bearer credential, one stable request identifier, and a replay outcome. The realtime result is similarly bounded: socket events improve timeliness, while REST and PostgreSQL retain authority. This trade-off favors consistency over the impression of fully offline or instant distributed behavior.

7.5.4 Analytics and Workload Result

The system converts stored work into descriptive summaries and explainable rule paths. This is appropriate for an academic prototype without a consented training dataset. Insight copy now describes workload patterns without diagnostic claims, and guidance scores are clamped to the interval from zero to one. The score is a bounded rule output, not a calibrated probability or validated health measure. The result is useful as reflective software because that boundary is visible to the user.

7.6 Current Limitations

Table 7.3 summarizes current limitations and their effect.

Table 7.3 Current limitations and their effect

Limitation	Current effect	Priority
Database and browser lanes not rerun	Final evidence is split between current and historical results	HIGH release gate
Production realtime origin policy not verified	Fail-closed room authorization is implemented, but deployed origin restrictions are not evidenced	HIGH before public deployment
Heuristic guidance scores are not calibrated	A bounded rule score must not be interpreted as probability or diagnosis	ONGOING transparency
Automated abuse rules use fixed thresholds	Rapid or repeated legitimate work can require moderator review	HIGH policy/governance
Health readiness checks configuration only	A configured but unavailable database may still be labelled configured	HIGH operations
Manual validation-only deploy workflow	No automatic tests, migrations, provider deployment, or rollback	HIGH operations
No provider-specific deployment manifest	Public production environment is not reproducibly defined	HIGH operations
No hosted email/push evidence	Delivery, retry, bounce, and reputation behavior are unknown	MEDIUM/HIGH
Chromium/virtual-authenticator emphasis	Cross-browser and physical credential behavior are unknown	MEDIUM

Limitation	Current effect	Priority
No formal usability/accessibility study	Ease of use and conformance cannot be claimed	MEDIUM
No load, soak, backup, or restore evidence	Capacity and recovery objectives are undefined	HIGH before real data
No git metadata in supplied workspace	Provenance and current remote CI status cannot be authenticated	MEDIUM evidence
Unfilled academic identity details	Cover, declaration, and acknowledgement retain clearly labelled fill-in blanks	REQUIRED before submission

7.7 Requirements Traceability Summary

The Chapter III requirement identifiers are traced throughout the design, implementation, testing, and result tables. This traceability shows that implementation and tests are concentrated around identity, quest completion, rewards, privileged access, and offline replay. It also makes absent evidence visible: database and browser lanes require a fresh isolated run; broad user experience needs participant evidence; and production operations need external infrastructure evidence.

- Implemented and current-pass: core validation, authentication rejection, isolated business rules, frontend offline/session utilities, type checking, lint, and builds.
- Implemented with current source/unit evidence but incomplete full-stack rerun: relational concurrency, rollback, registration/onboarding, offline replay, account control, and privileged browser workflow.
- Implemented but needing expanded verification: broader focus, guild, marketplace, safety, analytics, insight, notification, and administrative journeys.
- Deferred: public deployment, provider delivery, physical authenticator, recovery drills, cross-browser/load/accessibility/usability evaluation.

7.8 Discussion

The result demonstrates that a final-year project can integrate modern web concerns without splitting the application into premature microservices. The modular monolith supports transactions, shared types within each package, isolated services, background jobs, and a single local operational topology. The complexity is nevertheless substantial. Each additional progression, social, delivery, or administrative feature creates authorization and consistency obligations that must be maintained beyond a successful demo.

The most important analytical lesson is to separate an implemented mechanism from an established outcome. LevelUpX implements rewards, but the literature does not allow me to claim universal motivation. It implements focus sessions, but elapsed time is not learning quality. It implements workload rules, but the output is not diagnosis. It implements monitoring configuration, but a historical dashboard is not current production availability. These distinctions make the result more defensible in examination and more useful for future engineering work.

CHAPTER VIII

CONCLUSION AND FUTURE WORK

Conclusion

I designed and implemented LevelUpX to connect planning, focused work, progression feedback, reflection, and controlled social accountability in one web application. The result includes verified identity, projects and quests, focus-session records, atomic rewards, ledgers, profile progression, achievements, skills, cosmetics, guilds, team quests, leaderboards, accountability, analytics, rule-based insights, notifications, abuse reporting, account controls, administration, background jobs, realtime refreshes, monitoring foundations, and a narrow offline replay workflow.

The project met its main technical aim as a substantial local implementation. The modular-monolith architecture kept business areas understandable while allowing PostgreSQL transactions to protect cross-feature consistency. Strict TypeScript, Zod validation, role and ownership middleware, session controls, safe error responses, idempotency, and layered tests provided a stronger foundation than a page-only prototype. The current non-database checks passed on 12 August 2026. Sixteen database tests were collected but skipped because no isolated test database was configured, and the browser lane was not rerun after the adherence changes.

I do not conclude that the project improves motivation, productivity, learning, or well-being for every user. The literature and current evidence do not support that statement. I conclude that the project implements a coherent, testable technical workflow that can be evaluated in future user research. I also conclude that public production use should wait until the remaining deployment, recovery, capacity, policy, and full-stack verification work is addressed.

8.1 Major Contributions

Table 8.1 summarizes major contributions.

Table 8.1 Major contributions

Contribution	What I implemented	Why it is useful
Integrated work loop	Quest preparation, focus record, completion reward, analytics, and next action	Connects previously separated productivity records
Transactional progression	Guarded completion with profile and ledger updates	Prevents partial or duplicate payout under tested conditions
Consistent progression rules	Atomic UTC streak maintenance, one difficulty adjustment, inflation-aware coins, and skill prerequisites	Keeps linked reward and unlock behavior explicit and testable
Bounded offline action	Credential-free IndexedDB queue with idempotent replay	Provides useful offline capture without unsafe economy reconciliation
Scoped social automation	Membership-gated guild rooms, accepted-peer focus presence, and recurring weekly team quests	Combines timeliness and collaboration with explicit authorization
Layered privileged access	Pre-provisioned trusted device, codes, passkey step-up, mutable device check	Raises assurance for high-impact administrator operations
Modular monolith	Domain routes, services, repositories, providers, jobs, and one relational model	Balances maintainability and deployability for the project scale
Transparent evidence	Current/historical test separation and explicit limitations	Makes the report reproducible and defensible
Explainable insight boundary	Non-diagnostic rule-based guidance with bounded scores	Avoids presenting an unvalidated model as medical or behavioral proof

Contribution	What I implemented	Why it is useful
Operational exports and safety signals	CSV/PDF administration exports plus automated rapid/repetitive completion reports	Supports reviewable evidence and moderator attention without automatic punishment

8.1.1 Lessons Learned

1. A reward is a financial-like consistency problem even when it has no cash value; balances and ledgers need atomic rules.
2. Authentication is not authorization. Every REST route, realtime subscription, scheduled action, and provider boundary needs its own access decision.
3. Offline behavior is safest when the mutation set is deliberately small and each action has a stable identity and visible failure state.
4. Real-time delivery should improve timeliness without becoming the only copy of important state.
5. Static checks, unit tests, database tests, and browser tests answer different questions and must not be reported as interchangeable.
6. A feature label can overclaim even when the algorithm is transparent; bounded scores and non-diagnostic wording make the intended interpretation explicit.
7. Operational configuration is not a production deployment until providers, secrets, migrations, recovery, monitoring, and rollback are verified together.
8. A credible academic report records evidence boundaries and incomplete work instead of hiding them behind broad success language.

8.2 Future Enhancements

Table 8.2 summarizes prioritized future enhancements.

Table 8.2 Prioritized future enhancements

Priority	Enhancement	Acceptance evidence
P0	Rerun database and Playwright lanes in isolated CI	Fresh artifacts linked to an authenticated commit

Priority	Enhancement	Acceptance evidence
P0	Restrict production Socket.IO origins and complete transport-level security review	Only approved HTTPS origins connect; guild and presence allow/deny cases pass
P0	Define and test a provider-specific staging deployment	Migration, smoke, rollback, and audit artifacts are retained
P1	Add live database and Redis readiness probes	Health becomes not-ready when a required dependency fails
P1	Configure managed secrets, TLS, database, Redis, backups, and restore drills	Documented recovery objectives and successful restoration
P1	Verify hosted email, push, alerts, and external uptime	Delivery/retry/bounce and alert-response evidence
P1	Add route-level authorization inventory and security review	No unexplained protected handler; findings resolved or accepted
P1	Review automated abuse thresholds and moderator policy with representative data	Measured false-positive/false-negative cases and documented appeal/retention rules
P2	Offer an optional user-timezone streak policy with a migration/backfill design	UTC and local-boundary unit/DB/browser cases remain deterministic
P2	Expose supported quest/project edits consistently in the UI	Browser workflows cover create, update, archive, and error states
P2	Expand focus, guild, marketplace, safety, analytics, and admin browser journeys	Deterministic critical-path suite with stable fixtures
P2	Add Firefox/WebKit, narrow devices, keyboard, screen-reader, and automated accessibility checks	Published compatibility and accessibility matrix
P2	Run load, soak, job-overlap, and cache-effect tests from a defined traffic model	Latency/error/capacity results with thresholds

Priority	Enhancement	Acceptance evidence
P3	Conduct a consented usability and outcome study	Defined sample, comparison, measures, ethics/consent, and honest analysis
P3	Evaluate user-controlled ranking privacy and opt-out	Requirements and participant feedback support the chosen control
P3	Consider provider-assisted insights only after governance and evaluation	Dataset/provenance, evaluation, bias, explanation, fallback, and deletion controls

8.2.1 Suggested Evaluation Plan

A future academic evaluation should use a defined research question rather than ask whether the entire application is simply good. One study could compare a plain task list with LevelUpX's quest and feedback loop over a fixed period. Measures should separate completion quantity, task quality, planning accuracy, voluntary continued use, perceived autonomy, interruption, and chart comprehension. The analysis should predefine exclusions and avoid treating usage as intrinsic motivation.

A technical evaluation should run independently. It should measure API latency under read and mutation mixes, concurrent completion protection, rate-limit behavior, job overlap, cache hit value, database pool pressure, browser compatibility, accessibility, backup restoration, and provider failure. Security review should include ownership, RBAC, session revocation, realtime subscription authorization, export/delete, WebAuthn origin configuration, secrets, logs, and administrative audit behavior.

Final Statement

LevelUpX gave me practical experience in requirements analysis, relational modeling, frontend and backend engineering, security boundaries, transactions, offline synchronization, realtime communication, background jobs, observability, automated testing, and evidence-based documentation. The project is ready for academic demonstration after the labelled student, guide, institution, place, date, and academic-year blanks are completed. It is not yet represented as a public production service. The next responsible step is to run the skipped database and browser lanes against an authenticated

revision, restrict the deployed realtime origin policy, and then proceed to a controlled staging deployment.

8.3 Bibliography

- [1] S. Deterding, D. Dixon, R. Khaled, and L. Nacke, "From game design elements to gamefulness: Defining gamification," in Proceedings of the 15th International Academic MindTrek Conference, 2011, pp. 9-15. doi: 10.1145/2181037.2181040.
- [2] M. Sailer, J. U. Hense, S. K. Mayr, and H. Mandl, "How gamification motivates: An experimental study of the effects of specific game design elements on psychological need satisfaction," *Computers in Human Behavior*, vol. 69, pp. 371-380, 2017. doi: 10.1016/j.chb.2016.12.033.
- [3] E. D. Mekler, F. Bruhlmann, A. N. Tuch, and K. Opwis, "Towards understanding the effects of individual gamification elements on intrinsic motivation and performance," *Computers in Human Behavior*, vol. 71, pp. 525-534, 2017. doi: 10.1016/j.chb.2015.08.048.
- [4] R. N. Landers, K. N. Bauer, and R. C. Callan, "Gamification of task performance with leaderboards: A goal setting experiment," *Computers in Human Behavior*, vol. 71, pp. 508-515, 2017. doi: 10.1016/j.chb.2015.08.008.
- [5] E. A. Locke and G. P. Latham, "Building a practically useful theory of goal setting and task motivation: A 35-year odyssey," *American Psychologist*, vol. 57, no. 9, pp. 705-717, 2002. doi: 10.1037/0003-066X.57.9.705.
- [6] R. M. Ryan and E. L. Deci, "Self-determination theory and the facilitation of intrinsic motivation, social development, and well-being," *American Psychologist*, vol. 55, no. 1, pp. 68-78, 2000. doi: 10.1037/0003-066X.55.1.68.
- [7] F. Biwer, W. Wiradhany, M. G. A. oude Egbrink, and A. B. H. de Bruin, "Understanding effort regulation: Comparing Pomodoro breaks and self-regulated breaks," *British Journal of Educational Psychology*, vol. 93, suppl. 2, pp. 353-367, 2023. doi: 10.1111/bjep.12593.
- [8] I. Li, A. K. Dey, and J. Forlizzi, "A stage-based model of personal informatics systems," in Proceedings of CHI 2010, 2010, pp. 557-566. doi: 10.1145/1753326.1753409.

- [9] E. Demerouti, A. B. Bakker, F. Nachreiner, and W. B. Schaufeli, "The Job Demands-Resources model of burnout," *Journal of Applied Psychology*, vol. 86, no. 3, pp. 499-512, 2001. doi: 10.1037/0021-9010.86.3.499.
- [10] World Health Organization, "Burn-out an occupational phenomenon: International Classification of Diseases," May 28, 2019. <https://www.who.int/news/item/28-05-2019-burn-out-an-occupational-phenomenon-international-classification-of-diseases> (accessed Aug. 12, 2026).
- [11] A. Russell, J. Song, J. Archibald, and M. Kruisselbrink, Eds., "Service Workers 1," W3C Candidate Recommendation, Nov. 19, 2019. <https://www.w3.org/TR/2019/CR-service-workers-1-20191119/> (accessed Aug. 12, 2026).
- [12] Socket.IO, "Delivery guarantees," Socket.IO 4.x Documentation, updated Jun. 9, 2026. <https://socket.io/docs/v4/delivery-guarantees/> (accessed Aug. 12, 2026).
- [13] Vercel, "Next.js Docs: App Router," updated Mar. 25, 2026. <https://nextjs.org/docs/app> (accessed Aug. 12, 2026).
- [14] React, "Your First Component." <https://react.dev/learn/your-first-component> (accessed Aug. 12, 2026).
- [15] Microsoft, "The TypeScript Handbook." <https://www.typescriptlang.org/docs/handbook/intro.html> (accessed Aug. 12, 2026).
- [16] Express.js, "Using middleware." <https://expressjs.com/en/guide/using-middleware/> (accessed Aug. 12, 2026).
- [17] Prisma, "Prisma ORM." <https://www.prisma.io/docs/orm> (accessed Aug. 12, 2026).
- [18] PostgreSQL Global Development Group, "PostgreSQL 16 Documentation." <https://www.postgresql.org/docs/16/index.html> (accessed Aug. 12, 2026).
- [19] Zod, "Basic usage." <https://zod.dev/basics> (accessed Aug. 12, 2026).
- [20] Redis, "Redis data types." <https://redis.io/docs/latest/develop/data-types/> (accessed Aug. 12, 2026).
- [21] MDN Web Docs, "Progressive web apps." https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps (accessed Aug. 12, 2026).

- [22] MDN Web Docs, "IndexedDB API." https://developer.mozilla.org/en-US/docs/Web/API/IndexedDB_API (accessed Aug. 12, 2026).
- [23] W3C, "Web Authentication: An API for Accessing Public Key Credentials - Level 2," W3C Recommendation, Apr. 8, 2021. <https://www.w3.org/TR/webauthn-2/> (accessed Aug. 12, 2026).
- [24] Docker, "Docker Compose documentation." <https://docs.docker.com/compose/> (accessed Aug. 12, 2026).
- [25] Vitest, "Getting Started." <https://vitest.dev/guide/> (accessed Aug. 12, 2026).
- [26] Microsoft, "Playwright introduction." <https://playwright.dev/docs/intro> (accessed Aug. 12, 2026).
- [27] OWASP Foundation, "Application Security Verification Standard 5.0.0," May 30, 2025. <https://github.com/OWASP/ASVS/releases/tag/v5.0.0> (accessed Aug. 12, 2026).
- [28] LevelUpX source workspace, Prisma schema, route definitions, tests, configuration, and project notes, unpublished academic project materials, accessed Aug. 12, 2026.
- [29] LevelUpX, "Local Observability Stack," PROJECT_LOG_2026-08_OBSERVABILITY_STACK.md, internal dated verification record, Aug. 5, 2026.

APPENDIX A

ORIGINAL SYNOPSIS CONFORMANCE

This appendix uses the six objectives in the original project synopsis as the baseline. It records where each objective is implemented and separates feature conformance from unavailable production or evaluation evidence. Detailed functional and non-functional requirements remain in Chapter III, module and relational behavior is expanded in Chapters IV and V, and the verification boundary is recorded in Chapter VI.

A.1 Objective-to-Implementation Matrix

Table A.1 summarizes original synopsis objectives mapped to the implemented system.

Table A.1 Original synopsis objectives mapped to the implemented system

ID	Original commitment	Principal implementation	Verdict and evidence boundary
O1	Responsive PWA with quests, a 25-minute Pomodoro-style preset, other focus presets, and explicit work/break sessions	Next.js/React routes; quest and focus modules; manifest/service worker; IndexedDB quick-capture queue	MATCHES. Frontend tests, static checks, and build pass; Playwright was not rerun and offline scope remains deliberately narrow.
O2	RPG progression with XP, coins, levels, classes, UTC streaks, prerequisite skills, achievements, cosmetics, and personal rewards	Locked quest-completion transaction; profile and ledgers; skill prerequisite graph; achievement, cosmetic, marketplace, and custom-reward modules	MATCHES LOCALLY. Rule tests pass; 16 relational cases were collected but skipped without an isolated test database.

ID	Original commitment	Principal implementation	Verdict and evidence boundary
O3	Guilds, authorized realtime rooms, leaderboards, accepted-peer presence, and idempotent recurring weekly team quests	GuildMember authorization; Socket.IO room gate; leaderboard; accountability presence/IDLE eviction; series/week recurrence uniqueness	MATCHES LOCALLY. Social/realtime/job tests pass; current browser, reconnect, origin-policy, and load evidence is pending.
O4	Descriptive analytics, heatmaps, reports, and bounded workload, recovery, consistency, and scheduling guidance	Personal analytics/CSV; administrative analytics and marketplace CSV/PDF; default rules; optional provider contracts; clamped insight scores; fixed safe BURNOUT_WARNING copy	MATCHES DEFAULT RULE SCOPE. Optional external text is not fully semantic-filtered; hosted provider governance, participant study, and ML validation remain deferred.
O5	Verified identity, RBAC, staged privileged login, moderation, automatic review signals, economy controls, and audit records	Registration/session modules; ADMIN/SUPER_ADMIN middleware; trusted device plus three codes plus WebAuthn; moderation, audit, exports, and rule-keyed OPEN abuse signals	MATCHES LOCALLY. Unit/HTTP/frontend evidence passes; hosted delivery, physical passkey, penetration review, and production policy remain deferred.

ID	Original commitment	Principal implementation	Verdict and evidence boundary
O6	Separate unit, HTTP integration, static, build, database, and browser verification lanes with unavailable lanes reported separately	Vitest/Supertest/Playwright configurations; strict type checking and linting; production builds; Prisma and Compose validation	MATCHES THE EVIDENCE BOUNDARY. Available lanes pass: 270 backend, 5 HTTP, and 10 frontend tests; DB was skipped and Playwright was not rerun.

Final conformance verdict: the current source and project report match the functional system proposed in the original synopsis. No synopsis-only product feature remains presented as complete without supporting implementation. The match is qualified rather than absolute because database and browser reruns, production deployment, hosted delivery, recovery, capacity, formal usability, and outcome evaluation are explicitly unfinished. These are verification and deployment gaps, not hidden substitutions for the proposed LevelUpX feature set.